# Rockchip Sensor Driver Debugging and Developer Guide

ID: RK-KF-YF-A56

Release Version: V1.1.1

Release Date: 2026-05-29

Security Level: □Top-Secret □Secret □Internal ■Public

**DISCLAIMER**

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

**Trademark Statement**

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

**All rights reserved. ©2026. Rockchip Electronics Co., Ltd.**

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, Software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

## Preface

### Overview

This document primarily introduces Camera driver debugging and feature development processes, common issues, and solutions on the Rockchip Linux platform.

### Product Versions

| Chip Models | Kernel Version |
| --- | --- |
| RV1109/RV1126 | Linux 4.19 |
| RK356X | Linux 4.19 |
| RK3588 | Linux 5.10 |
| RK3576 | Linux 5.10 |
| RV1106 / RV1103 | Linux 5.10 |
| RV1106B / RV1103B | Linux 5.10 |
| RV1126B | Linux 6.1 |

### Intended Audience

This document (this guide) is mainly intended for:

Technical Support Engineers

Software Development Engineers

### Revision Record

| Version | Author | Date | Modification Description |
| --- | --- | --- | --- |
| V1.0.0 | MLC | 2025-08-10 | Initial version |
| V1.1.0 | SYF | 2025-09-28 | Added MCU Quick Start Configuration Guide |
| V1.1.1 | ZLX | 2026-05-29 | Fixed typos |

Table of Contents


Rockchip Sensor Driver Debugging and Developer Guide

- 1. Preparation Before Debugging
  - 1.1 Confirming SoC Specifications
  - 1.2 Sensor Datasheet
  - 1.3 Sensor Initialize Settings
- 2. Driver Development
  - 2.1 Writing Driver Code
    - 2.1.1 Key Parameters of the Sensor
    - 2.1.2 Key Parameters of the Sensor Registers
      - 2.1.2.1 chip_id
      - 2.1.2.2 Control Mode
      - 2.1.2.3 Exposure reg
      - 2.1.2.4 Gain Reg
      - 2.1.2.5 Group Hold
      - 2.1.2.6 Test Pattern
        - 2.1.2.6.1 The Purpose and Function of Test Pattern
        - 2.1.2.6.2 Common Test Pattern Types
      - 2.1.2.7 VTS
      - 2.1.2.8 Mirror / Flip
  - 2.2 Driver Framework and Introduction
    - 2.2.1 Data Type Introduction
      - 2.2.1.1 struct i2c_driver
      - 2.2.1.2 struct v4l2_subdev_ops
      - 2.2.1.3 struct v4l2_subdev_core_ops
      - 2.2.1.4 struct v4l2_subdev_video_ops
      - 2.2.1.5 struct v4l2_subdev_pad_ops
      - 2.2.1.6 struct v4l2_ctrl_ops
    - 2.2.2 API Introduction
      - 2.2.2.1 xxxx_set_fmt
      - 2.2.2.2 xxxx_get_fmt
      - 2.2.2.3 xxxx_enum_mbus_code
      - 2.2.2.4 xxxx_enum_frame_sizes
      - 2.2.2.5 xxxx_g_frame_interval
      - 2.2.2.6 xxxx_s_stream
      - 2.2.2.7 xxxx_runtime_resume
      - 2.2.2.8 xxxx_runtime_suspend
      - 2.2.2.9 xxxx_set_ctrl
      - 2.2.2.10 xxx_enum_frame_interval
      - 2.2.2.11 xxxx_g_mbus_config
      - 2.2.2.12 xxxx_get_selection
      - 2.2.2.13 xxxx_ioctl
  - 2.3 Driver Porting Steps
    - 2.3.1 Implementing Standard I2C Sub-device Driver
    - 2.3.2 Implementing v4l2 Sub-device Drivers
  - 2.4 Modifying Makefile
  - 2.5 Kconfig Modifications
- 3. Quick Start
  - 3.1 Concepts
  - 3.2 Driver-Related
    - 3.2.1 Kernel Drivers
    - 3.2.2 MCU Driver

- 3.3 MCU Quick Start Configuration Guide
  - 3.3.1 MCU Driver Framework and Introduction
    - 3.3.1.1 Data Type Description
      - 3.3.1.1.1 struct rk_camera_ops
      - 3.3.1.1.2 struct xxx_mode
      - 3.3.1.1.3 Struct xxx_dev
    - 3.3.1.2 API Description
      - 3.3.1.2.1 rk_xxx_control
      - 3.3.1.2.2 rk_xxx_init
      - 3.3.1.2.3 xxx_stream_on
      - 3.3.1.2.4 xxx_stream_off
      - 3.3.1.2.5 rk_xxx_get_expinf
      - 3.3.1.2.6 rk_xxx_set_expval
      - 3.3.1.2.7 rk_xxx_set_vts
      - 3.3.1.2.8 rk_xxx_get_input_fmt
      - 3.3.1.2.9 rk_xxx_set_input_fmt
      - 3.3.1.2.10 rk_xxx_match_dst_config
      - 3.3.1.2.11 rk_xxx_set_flip_mirror
      - 3.3.1.2.12 xxx_stream_on_late
      - 3.3.1.2.13 xxx_quick_stream
      - 3.3.1.2.14 rk_xxx_check_sensor_id
  - 3.3.2 Camera Driver Porting
    - 3.3.2.1 Adding Compilation Configuration Options
    - 3.3.2.2 Register Sequence Conversion
      - 3.3.2.2.1 Sequence Conversion Steps
    - 3.3.2.3 Configuring Basic Functions
  - 3.3.3 MCU Board-Level Configuration Guide
    - 3.3.3.1 CIS (CMOS Image Sensor) Device Registration
      - 3.3.3.1.1 struct gpio_desc
      - 3.3.3.1.2 camera_board_desc Structure Description
      - 3.3.3.1.3 `cif_hw_config` Function Description
      - 3.3.3.1.4 Single Camera Registration
  - 3.3.4 How to Add Board-Level Configuration
- 4. New Feature Support
  - 4.1 AOV
    - 4.1.1 Scenario Description
    - 4.1.2 Workflow
    - 4.1.3 Concept Introduction
      - 4.1.3.1 Hardware Standby Mode
      - 4.1.3.2 Software Standby Mode
    - 4.1.4 Driver Implementation
    - 4.1.5 AOV Reference Documentation
  - 4.2 Synchronization Mode (Master-Slave Mode)
    - 4.2.1 Sensor's Slave Mode
    - 4.2.2 Driver Implementation
  - 4.3 Flashlight Control
    - 4.3.1 Driver Implementation
    - 4.3.2 Reference Documents
  - 4.4 Resolution Switching
    - 4.4.1 Driver Implementation
  - 4.5 Compatibility of Multi-Lane Configurations
    - 4.5.1 Driver Implementation
- 5. FAQ

- 5.1 Sensor Not Recognized, Probe Failure, I2C Communication Failure
  - 5.1.1 Measuring MCLK and VDD Voltages Unsuccessfully
  - 5.1.2 Verify if the Power-Up Sequence of the Sensor Meets the Requirements
- 5.2 What should be the default values for `exp_def`, `hts_def`, and `vts_def` in the driver?
- 5.3 What should be the values for `link_freq` and `pixel_rate`
- 5.4 How to Verify if the Sensor is Successfully Brought up
- 5.5 Sensor Streaming with CRC/ECC Errors
  - 5.5.1 Confirming Error Sources and Log Details
  - 5.5.2 Software Configuration Check (Driver and DTS)
  - 5.5.3 Hardware Link Troubleshooting (Physical Layer Issues Account for Over 90%!)
  - 5.5.4 System-Level Verification and Debugging
  - 5.5.5 Emergency Situation Handling Strategy
  - 5.5.6 Typical Repair Measures
  - 5.5.7 Precautions for Sensor Index

6. AOV Reference Documents

7. Appendix
- 7.1 CIS Driver V4L2-controls List
- 7.2 MEDIA_BUS_FMT List
- 7.3 Camera Reference Driver List

# 1. Preparation Before Debugging

Before debugging the Camera driver, the following materials need to be prepared:

## 1.1 Confirming SoC Specifications

Ensure that the SoC platform used by the current Sensor supports the maximum resolution, the upper limit of input frequency, the highest rate per Lane, the number of supported Lanes, and the type of data input interface.

## 1.2 Sensor Datasheet

Obtain the Sensor Datasheet provided by the vendor of the related Sensor to facilitate the review of relevant information:

- Confirm the hardware voltage levels and power-up and power-down timing sequences.
- Confirm the Sensor Chip ID, which is used to check when Sensor probing.
- Confirm how to set exposure time and gain, and how to modify the frame rate.
- Confirm whether Slave mode is supported, how to connect multiple cameras, and how to configure related registers.
- Whether test mode is supported.
- Confirm whether high dynamic range (HDR) mode is supported.
- Confirm how to configure Mirror / Flip.

## 1.3 Sensor Initialize Settings

Obtain the Sensor register configuration list.

Typically provided by the Sensor vendor. Clarify specific requirements, such as resolution, frame rate, lane count, rate per lane, VTS, HTS, bit width, master clock frequency, etc.

At least prepare a standard resolution sequence. Depending on the actual project requirements, determine whether to prepare special resolution sequences such as binning, crop, etc.

For example, the register configuration file of SmartSens, cleaned_0x2b_SC530AI_27Minput_1080Mbps_2lane_10bit_1920x1080_60fps.ini, provides relevant information in the file name:

Sensor Name: SC530AI

Master clock provided to Sensor MCLK: 27MHz Sensor input to master control MIPI rate per Lane: 1080Mbps Number of Lanes used: 2 Bit width: 10bit Resolution: 1920x1080 (1080p) Frame rate: 60fps

Below is the template for requesting configuration from sensor manufacturers: Select specific entries based on actual project conditions.

```
sensor: XXXX
mclk: 24MHz/27MHz/37.5MHz
mode: raw8/raw10/raw12/raw16
output: 720p/1080p/1440p/... (resolution)
interface: mipi 1/2/4 Lane
frame rate: 30/60/120 fps
master control platform: RK3588/RV1106/RV1103B/RV1126B/...
project (product) form: doorbell/doorbell/IPC/... (optional)
```

# 2. Driver Development

## 2.1 Writing Driver Code

When writing a new Sensor driver, it can be based on a Sensor driver with similar specifications (`kernel/drivers/media/i2c/`) or modified from a driver of the same manufacturer with a similar model.

The following sections take the development of the SC200AI Sensor driver from SmartSens as an example to introduce the process of Sensor driver development on the RK platform.

### 2.1.1 Key Parameters of the Sensor

The `Sensor mode` structure defines the important parameters of the Sensor, and accurate parameter information must be filled in for different resolutions.

```
struct XXXX_mode {
    u32 bus_fmt;    // Output pixel format
    u32 width;      // Output width
    u32 height;     // Output height
    struct v4l2_fract max_fps; // Maximum output frame rate
    u32 hts_def;    // Default line total period
    u32 vts_def;    // Default frame total period
    u32 exp_def;    // Default exposure value
    u32 mipi_freq_idx; // Link frequency index value
    u32 bpp;        // Bits per pixel
    const struct regval *reg_list; // Register list
    u32 hdr_mode;   // Wide dynamic range mode
    u32 vc[PAD_MAX]; // Virtual channel array
};
```

The `supported_modes` structure array saves the mode structures of the supported resolutions, which are used to traverse and match during the probe initialization or when switching resolutions.

Taking SC200AI as an example:

```
static const struct sc200ai_mode supported_modes[] = {
    { // 1080p 30fps Linear configuration parameters
        .width = 1920,
        .height = 1080,
        .max_fps = {
            .numerator = 10000,
            .denominator = 300000,
        },
        .exp_def = 0x0080,
        .hts_def = 0x44C * 2,
        .vts_def = 0x0465,
        .bus_fmt = MEDIA_BUS_FMT_SBGGR10_1X10,
        .reg_list = sc200ai_linear_10_1920x1080_30fps_regs,
        .hdr_mode = NO_HDR,
        .bpp = 10,
        .mipi_freq_idx = 1,
        .vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_0,
    },
    { // 540p 120fps Linear configuration parameters
        .width = 960,
        .height = 540,
        .max_fps = {
            .numerator = 10000,
            .denominator = 1200000,
        },
        .exp_def = 0x0080,
        .hts_def = 0x420,
        .vts_def = 0x249,
        .bus_fmt = MEDIA_BUS_FMT_SBGGR10_1X10,
        .reg_list = sc200ai_linear_10_960x540_120fps_regs,
        .hdr_mode = NO_HDR,
        .bpp = 10,
        .mipi_freq_idx = 0,
        .vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_0,
    },
    { // 1080p 30fps Wide dynamic range configuration parameters
        .width = 1920,
        .height = 1080,
        .max_fps = {
            .numerator = 10000,
            .denominator = 300000,
        },
        .exp_def = 0x0080,
        .hts_def = 0x44C * 2,
        .vts_def = 0x08CC,
        .bus_fmt = MEDIA_BUS_FMT_SBGGR10_1X10,
        .reg_list = sc200ai_hdr_10_1920x1080_regs,
        .hdr_mode = HDR_X2,
        .bpp = 10,
        .mipi_freq_idx = 1,
        .vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_1,
        .vc[PAD1] = V4L2_MBUS_CSI2_CHANNEL_0,//L->csi wr0
        .vc[PAD2] = V4L2_MBUS_CSI2_CHANNEL_1,
```

```
        .vc[PAD3] = V4L2_MBUS_CSI2_CHANNEL_1,//M->csi wr2
    },
};
```

Parameter Description:

- `bus_fmt` is the pixel arrangement format output by the Sensor, generally indicating how many bits (e.g., 10-bit), BGGR is the pixel arrangement order, i.e., the Bayer format. An incorrect Bayer order does not affect the output image (but may cause color bias).
- `width`, `height` are the effective width and height output by the Sensor, which are the most important parameters and are generally less than `hts` and `vts`.
- `max_fps` is the maximum frame rate supported by the current mode, calculated as frame rate = denominator / numerator. The numerator generally remains unchanged; for an accurate 30 frames, the denominator is 300000.
- `exp_def` is the default exposure value (filled in according to the value defined by the initialization register).
- `hts_def` is the total number of points in a default line (filled in according to the value defined by the initialization register).
- `vts_def` is the total number of lines in a default frame (filled in according to the value defined by the initialization register). This value should not be incorrect, as it affects the output frame rate.
- `global_reg_list` is the common initialization register of the Sensor. If not available, it can be empty.
- `reg_list` is the register sequence provided by the Sensor manufacturer.
- `hdr_mode` corresponds to the HDR mode of the current parameters, NO_HDR indicates linear mode, HDR_X2 / HDR_X3 indicates wide dynamic range, using 2 or 3 frames for blending.
- `mipi_freq_idx` corresponds to the frequency index in the `link_freq_menu_items` array below. Generally, the manufacturer provides the rate per Lane in Mbps. Due to the dual-edge sampling of MIPI, the MIPI frequency is half of the sampling rate. For example, in the first linear sequence, the manufacturer's sequence indicates a rate of 720Mbps, and the MIPI frequency is 360M, which means the `link_freq` link frequency is 360M. Important Note: Incorrect parameters can easily lead to failure in receiving images, so confirmation is essential.
- `bpp` indicates the number of bits occupied by each pixel.

**Note:**

The difference between linear and wide dynamic range configurations is in two places, one is the difference in `hdr_mode`, and the other is the difference in the `vc` virtual channel array.

- Linear: hdr_mode = NO_HDR, vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_0;
- Wide Dynamic Range: hdr_mode = HDR_X2/HDR_X3, vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_0; vc[PAD1] = V4L2_MBUS_CSI2_CHANNEL_0; vc[PAD2] = V4L2_MBUS_CSI2_CHANNEL_1; vc[PAD3] = V4L2_MBUS_CSI2_CHANNEL_1;

## 2.1.2 Key Parameters of the Sensor Registers

Below is a simple list and introduction of some important registers that may be used in Sensor driver development. There may be some differences depending on the Sensor manufacturer and model.

#### 2.1.2.1 chip_id

Sensor Chip ID, serves as the identification card for each Sensor, used to recognize whether the Sensor is correctly loaded when the driver is loaded. The Sensor probe interface will check the Chip ID for each chip; if the Sensor Chip ID is read, it is considered that the Sensor device is recognized normally, I2C read/write communication is normal, and all voltages are normal.

```
#define CHIP_ID                 0xcb1c
#define SC200AI_REG_CHIP_ID     0x3107
```

**Note**:

For some Sensors that do not have a Chip ID, and thus cannot verify whether the Sensor device is normally recognized and mounted, a specific (or several) read-only registers can be selected to read, and the default value of the read result can be used to confirm.

#### 2.1.2.2 Control Mode

The control register for starting and stopping the flow of the Sensor, generally after all Sensor registers have been successfully written, writing to this register can control the Sensor to start or stop the flow.

```
#define SC200AI_REG_CTRL_MODE       0x0100
#define SC200AI_MODE_SW_STANDBY     0x0
#define SC200AI_MODE_STREAMING      BIT(0)
```

**Note**:

Please do not configure the value of this register in the register sequence, we use the start and stop flow interfaces to control the writing of this register separately.

#### 2.1.2.3 Exposure reg

Exposure Control Register, depending on linear/wide dynamic range mode, different exposure control registers may be configured, as specified in the Sensor manual.

```
#define SC200AI_REG_EXPOSURE_H      0x3e00  // Long exposure high byte
#define SC200AI_REG_EXPOSURE_M      0x3e01  // Long exposure middle byte
#define SC200AI_REG_EXPOSURE_L      0x3e02  // Long exposure low byte
#define SC200AI_REG_SEXPOSURE_H     0x3e22  // Short exposure high byte
#define SC200AI_REG_SEXPOSURE_M     0x3e04  // Short exposure middle byte
#define SC200AI_REG_SEXPOSURE_L     0x3e05  // Short exposure low byte
#define SC200AI_EXPOSURE_MIN        1       // Minimum exposure line, generally not changed
unless the sensor has a minimum exposure limit
#define SC200AI_EXPOSURE_STEP       1       // Exposure step, generally not modified
```

**Note**:

- Some Sensor's exposure registers may exceed 16bit (for example, high byte + low byte, such as here it is 24bit, with high, middle, and low bytes), requiring multiple writes. It may also be for long frames or short frames, which need to be written according to the actual situation.

- Gain values may need to be calculated according to specific formulas, for example:
  - Analog gain may be a linear value or may involve looking up a table;
  - Digital gain may be a floating-point multiplier, but what is written is a fixed-point number (such as Q8.8 format).

#### 2.1.2.4 Gain Reg

Gain control register, divided into analog gain control register and digital gain control register.

Each type of gain control register sometimes includes coarse gain (Coarse gain, large-grained, generally corresponding to the integer part of the gain, used for rough adjustment of gain) and fine gain (Fine gain, generally corresponding to the fractional part of the gain, used for fine-tuning of gain).

Below is a brief introduction to the related concepts:

**Analog Gain**

- Amplifies the voltage signal output by the sensor in the **analog signal domain** (before ADC);
- Advantage: Relatively less impact on image noise (compared to digital gain);
- Disadvantage: Limited amplification factor, excessive amplification can introduce more noise and nonlinear distortion;
- Corresponding register: `ANA_GAIN` (coarse), `ANA_FINE_GAIN` (fine-tuning)

**Digital Gain**

- Multiplies the original digital values of the image (such as 10bit/12bit data) in the **digital signal domain** (after ADC);
- Advantage: Simple implementation, flexible;
- Disadvantage: Essentially multiplying digital values, **amplifies noise and quantization errors**, especially prone to color noise in low light;
- Corresponding register: `DIG_GAIN` (coarse), `DIG_FINE_GAIN` (fine-tuning)

**What is the target of gain adjustment?**

- To enhance picture brightness in **low-light environments** (by increasing gain);
- In automatic exposure (AE) algorithms, **dynamically adjust gain and exposure time (shutter/line count)** to achieve target brightness;
- Control **image signal-to-noise ratio (SNR), dynamic range**, etc.

```
#define SC200AI_REG_DIG_GAIN        0x3e06  //Digital main gain
#define SC200AI_REG_DIG_FINE_GAIN   0x3e07  //Digital fine gain
#define SC200AI_REG_ANA_GAIN        0x3e08  //Analog main gain
#define SC200AI_REG_ANA_FINE_GAIN   0x3e09  //Analog fine gain
#define SC200AI_REG_SDIG_GAIN       0x3e10  //Short frame digital main gain
#define SC200AI_REG_SDIG_FINE_GAIN  0x3e11  //Short frame digital fine gain
#define SC200AI_REG_SANA_GAIN       0x3e12  //Short frame analog main gain
#define SC200AI_REG_SANA_FINE_GAIN  0x3e13  //Short frame analog fine gain
#define SC200AI_GAIN_MIN            0x0040  //Minimum gain value
#define SC200AI_GAIN_MAX            (54 * 32 * 64)       // Maximum gain value, approximate
value for integer arithmetic
#define SC200AI_GAIN_STEP           1   //Gain step
#define SC200AI_GAIN_DEFAULT        0x0800  //Default gain value
```

```
#define SC200AI_LGAIN               0   //Long frame gain
#define SC200AI_SGAIN               1   //Short frame gain
```

Note:

When supporting HDR high dynamic mode, when setting Sensor gain, it may be necessary to set long frame and short frame gains, at this time, SC200AI_LGAIN, SC200AI_SGAIN are defined to distinguish.

#### 2.1.2.5 Group Hold

**Group Hold** is a mechanism in CMOS image sensors used for **synchronizing the update of exposure (shutter), gain, and other parameters**. The primary purpose is:

> To **ensure that all relevant parameters (such as analog gain, digital gain, row exposure time, etc.) remain consistent during the acquisition of a single frame of image, preventing parameter changes midway that could cause image anomalies (such as vertical stripes, uneven brightness, partial overexposure/underexposure, etc.)**.

Specifically: In CMOS sensors, many critical parameters (such as exposure time, analog/digital gain, etc.) are configured by writing to registers;

However, modifications to these registers **usually do not take effect immediately**, but rather **take effect at the start of the next frame/row cycle**; if you **modify these parameters during the acquisition of a single frame of image**, it could lead to **parts of the image using old parameters and other parts using new parameters**, resulting in a noticeable boundary line or anomaly; **Group Hold is used to "freeze" the timing of these parameter updates**, ensuring they take effect uniformly at a specified moment, thus ensuring the consistency of parameters within a single frame of image.**

```
#define SC200AI_REG_GROUP_HOLD      0x3812 // Group Hold control register
#define SC200AI_GROUP_HOLD_START    0x00  // Enable Group Hold (freeze parameter updates)
i.e., Group Hold Start
#define SC200AI_GROUP_HOLD_END      0x30  // Disable Group Hold (release parameter updates)
i.e., Group Hold End
```

#### 2.1.2.6 Test Pattern

Test pattern registers refer to the capability of **image sensors (such as CMOS/CCD Sensors) to output fixed patterns generated internally by the sensor (e.g., colored stripes, gradients, checkerboards, monochrome blocks, etc.) instead of capturing real optical images in a specific mode.**

In simpler terms, it means making the camera **not capture real scenes but output a "fake image" generated internally by the sensor** for testing, debugging, or verification purposes.

These patterns typically include:

- Color Bars
- Gradient
- Checkerboard
- Monochrome (pure red / green / blue / black / white)
- Grid

- Fixed Pattern Noise (FPN)

#### 2.1.2.6.1 The Purpose and Function of Test Pattern

1. **Sensor Functional Verification/Debugging**

   In the **Sensor Development/Verification Phase**, engineers can quickly confirm through Test Pattern:

   - Whether the Sensor is outputting image data normally
   - Whether the MIPI CSI-2 interface is working properly (whether the data path is OK)
   - Whether the image resolution and color format (such as RAW, RGB, YUV) are correct
   - Whether ISP or downstream modules can correctly receive and process the data stream

   For example:

   > You just powered up a Sensor and don't know if it's working properly, nor do you want to connect to a real lens/light source. **After enabling Test Pattern, if you can see colored stripes/gradient patterns on the screen or debugging tools, it indicates that the Sensor and data path are basically OK!**

2. **ISP Debugging and Algorithm Verification**

During the development phase of ISP (Image Signal Processor), Test Pattern can be used to:

- Verify the functionality of **ISP modules** (such as Demosaic, 3A, Gamma, CSC, Sharpening, etc.)
- Test the processing effects of ISP under **different input patterns**
- Check whether **color reproduction, brightness, contrast, noise suppression algorithms** are working normally

  Because Test Pattern is a known and controllable input, it is very suitable for **quantitative analysis and algorithm tuning**.

  For example: You input a **solid color block (like all red)** to see if the ISP output is also pure red, which can verify whether the **white balance/color correction module is abnormal**; input **gradient patterns** can be used to test **Gamma, dynamic range, tone mapping** and other functions.

3. **Camera Module Debugging & Calibration**

In the **camera module production/calibration phase**, Test Pattern can be used for:

- Verifying the data path of lens module + Sensor + ISP
- Simulating tests for autofocus (AF), auto exposure (AE), and auto white balance (AWB) algorithms
- Conducting basic functional tests in environments without real light sources

4. **Troubleshooting/Anomaly Analysis**

When the camera output image is abnormal (such as color bias, screen tearing, stripes, all black/white), you can:

- First, enable Test Pattern to see if it is still abnormal
  - If **Test Pattern output is normal** → it indicates that the problem may be in the **optical part (such as lens contamination, no light) or real scene input**
  - If **Test Pattern output is also abnormal** → it indicates that the problem is in **Sensor, MIPI path, ISP, or power supply**

So Test Pattern is a very important **troubleshooting tool** to distinguish whether the problem is in the **optical link** or **electrical signal/data processing link**.

5. **Factory Production Testing**

In the factory mass production testing phase of cameras/Sensors, Test Pattern can be used for:

- Automating the testing of whether Sensor output is normal
- Checking for dead pixels (Dead Pixel), hot pixels (Hot Pixel), and fixed noise
- Verifying data integrity, color accuracy, dynamic range, and other indicators

#### 2.1.2.6.2 Common Test Pattern Types

| Test Pattern Type | Description | Usage |
|---|---|---|
| **Solid Color (Single Color)** | The entire image is a solid color (such as pure red / green / blue / black / white) | Check color channels, dead pixels |
| **Color Bars (Colored Stripes)** | Similar to TV test patterns with colored horizontal stripes (red/green/blue/black/white combinations) | Verify color output, ISP processing |
| **Gradient (Gradual Change)** | Smooth transition from black to white or red-green-blue | Test dynamic range, Gamma, color mapping |
| **Checkerboard (Checkered Pattern)** | Alternating black and white square pattern | Test sharpness, edge processing, compression algorithms |
| **Moving Pattern (Dynamic Pattern)** | Some Sensors support dynamically changing test patterns | Test timing, buffering, latency |
| **Random / Noise Pattern** | Simulate noise patterns | Test noise suppression algorithms |
| **Cross Hatch / Grid** | Crosshatch grid pattern | Check geometric distortion, alignment |

```
#define SC200AI_REG_TEST_PATTERN                0x4501 // Test pattern control register
#define SC200AI_TEST_PATTERN_BIT_MASK   BIT(3) // Test pattern enable mask
```

### 2.1.2.7 VTS

In the context of **CMOS Image Sensors (CIS)** or **camera modules**, **HTS** and **VTS** are two very crucial timing parameters. This section focuses on VTS.

**VTS** stands for **Vertical Total Time** or **Vertical Total Lines**. Commonly referred to in Chinese as: **frame total cycle/frame total lines**. It represents the total number of lines (or total time) used by the sensor to output a complete frame of image (including all effective lines + blanking lines). It is usually the number of lines (lines), and can also be converted into time (such as ms), depending on the frame rate and VTS.

The content includes:

1. **Active Lines (number of effective lines)**: That is, the number of lines of actual image data, such as 1080 (for the height in 1920x1080 resolution).
2. **VSYNC (field synchronization signal)**
3. **Vertical Blanking Area (Vertical Blank / Front Porch/Back Porch)**: Used for sensor frame switching, internal processing, etc.

**VTS = Number of effective lines + Number of vertical blanking lines**

The relationship between HTS, VTS, image resolution, and frame rate:

1. Calculation of frame rate (FPS)

   The key formula for calculating frame rate is:

$$FPS = PCLK / (HTS \times VTS)$$

Where:

- **PCLK**: Pixel Clock, that is, the clock frequency of each pixel (usually in MHz or Hz), indicating how many pixel clocks are transmitted per second.
- **HTS**: The total number of clock cycles per line (determines how long a line takes)
- **VTS**: The total number of lines in a frame (determines how many lines a frame takes)

Therefore, **HTS and VTS together determine the frame rate of the sensor!** If you want to **increase the frame rate (FPS↑)**, you can:

- Reduce **VTS (total number of lines, i.e., reduce vertical blanking)**
- Or increase **PCLK (pixel clock)**

Conversely, if you want to reduce the frame rate (for example, longer exposure is needed in low light), you can increase VTS or decrease PCLK.

**The core difference between HTS and VTS**:

| Item | HTS (Horizontal Total) | VTS (Vertical Total) |
| --- | --- | --- |
| **Direction** | Horizontal direction (within a line) | Vertical direction (within a frame) |
| **Meaning** | Total clock cycles per line (including effective pixels + blanking) | Total number of lines per frame (including effective lines + blanking) |
| **Contents** | Effective pixels + HSYNC + Horizontal blanking area | Effective lines + VSYNC + Vertical blanking area |
| **Impact** | Line frequency, data transfer timing | Frame rate, frame time |
| **Relationship with frame rate** | FPS = PCLK / (HTS × VTS) | Increasing VTS will decrease frame rate, decreasing will increase |
| **Unit** | Number of clock cycles (or pixel clock counts) | Number of lines (lines) |
| **Typical Configuration Location** | Sensor register (line timing) | Sensor register (frame timing) |

```
#define SC200AI_REG_VTS_H               0x320e
#define SC200AI_REG_VTS_L               0x320f
```

#### 2.1.2.8 Mirror / Flip

Mirror and Flip control registers.

In the context of **Image Sensors (Sensors)**, **Mirror** and **Flip** are two functions used to control the **output direction or orientation** of images, affecting the **left-right or top-bottom direction of the image content output by the Sensor**, which falls under the category of **image geometric transformations**.

1. **Mirror** refers to the horizontal left-right symmetrical flip of the image, similar to the effect seen when a person looks into a mirror. Just as your left side appears on the right in the mirror when you look into it, this is **horizontal mirroring (Mirror)**.

The role in Sensor:

- By setting the **Mirror bit (or Horizontal Flip)** in the Sensor register, the image output by the Sensor can be flipped horizontally.
- Commonly used for:
    - **Sensor installed horizontally in reverse**, causing the image to be left-right reversed
    - Lens installation direction requiring horizontal flipping of the image

2. **Flip** refers to the vertical top-bottom symmetrical flip of the image, which is to say, inverting the image **upside down**. Just as you rotate a photo **180° upside down**, or look at the image from the ceiling, this is the effect of **vertical flipping (Flip)**.

The role in Sensor:

- By setting the **Flip bit (or Vertical Flip)** in the Sensor register, the image output by the Sensor can be flipped vertically.
- Commonly used for:
  - **Sensor installed upside down** (for example, the camera is installed upside down in the device)
  - Adjusting the image direction to match display requirements

**Combined effects of Mirror and Flip**

| Setting | Effect | Actual Performance |
| --- | --- | --- |
| **No Mirror / No Flip** | Normal Output | The image is not flipped, consistent with what the human eye sees (default direction) |
| **Mirror (Horizontal Mirror)** | Left-Right Flip | The image is left-right reversed, like looking into a mirror |
| **Flip (Vertical Flip)** | Up-Down Flip | The image is upside down, like an inverted photo |
| **Mirror + Flip** | Left-Right Mirror + Up-Down Flip = Rotate 180° | The image looks as if it has been rotated 180 degrees (complete reversal of top and bottom, left and right) |

In most CMOS image sensors (such as **Sony, OmniVision, SmartSens, GalaxyCore, Onsemi, etc.**), **Mirror and Flip functions are usually configured through register settings**.

```
#define SC200AI_FLIP_MIRROR_REG         0x3221 // Mirror and Flip control register

#define SC200AI_FETCH_MIRROR(VAL, ENABLE)       (ENABLE ? VAL | 0x06 : VAL & 0xf9)//Set
mirror
#define SC200AI_FETCH_FLIP(VAL, ENABLE)         (ENABLE ? VAL | 0x60 : VAL & 0x9f)//Set flip
```

3. v4l2-ctl command settings for mirror and flip

```
// The /dev/v4l-subdev2 node corresponds to the Sensor sub-device node. Refer to section
5.4
v4l2-ctl --set-ctrl=horizontal_flip=1 -d /dev/v4l-subdev2 # Enable horizontal mirror
v4l2-ctl --set-ctrl=horizontal_flip=0 -d /dev/v4l-subdev2 # Disable horizontal mirror

v4l2-ctl --set-ctrl=vertical_flip=1 -d /dev/v4l-subdev2 # Enable vertical flip
v4l2-ctl --set-ctrl=vertical_flip=0  -d /dev/v4l-subdev2 # Disable vertical flip

v4l2-ctl --set-ctrl=horizontal_flip=1 --set-ctrl=vertical_flip=1 -d /dev/v4l-subdev2  #
Enable both mirror and flip
v4l2-ctl --set-ctrl=horizontal_flip=0 --set-ctrl=vertical_flip=0  -d /dev/v4l-subdev2#
Disable both mirror and flip
```

## 2.2 Driver Framework and Introduction

The Camera Sensor communicates with the main controller via I2C, and the current Sensor driver is implemented as an I2C device driver. The Sensor driver also uses the v4l2 subdev approach to interact with the host driver.

## 2.2.1 Data Type Introduction

#### 2.2.1.1 struct i2c_driver

**Description**

Defines I2C device driver information

**Definition**

```
struct i2c_driver {
        ...

        /* Standard driver model interfaces */
        int (*probe)(struct i2c_client *client, const struct i2c_device_id *id);
        int (*remove)(struct i2c_client *client);
        ...
        struct device_driver driver;
        const struct i2c_device_id *id_table;
        ...
};
```

**Key Members**

| Member Name | Data Type | Role Description |
| --- | --- | --- |
| `probe` | `int (*)(struct i2c_client *,const struct i2c_device_id *)` | Device detection function, called when the driver matches the device |
| `remove` | `int (*)(struct i2c_client *)` | Device removal cleanup function |
| `driver` | `struct device_driver` | **Core driver structure, the .probe function is only called when the compatible field in the of_match_table matches the compatible field in the DTS file** |
| `id_table` | `const struct i2c_device_id *` | **Device matching table, if the kernel does not use of_match_table and DTS to register devices for matching, the kernel uses this table for matching** |

**Example**

```
#if IS_ENABLED(CONFIG_OF)
static const struct of_device_id sc200ai_of_match[] = {
        { .compatible = "smartsens,sc200ai" },
        {},
};
MODULE_DEVICE_TABLE(of, sc200ai_of_match);
#endif

static const struct i2c_device_id sc200ai_match_id[] = {
        { "smartsens,sc200ai", 0 },
        { },
};

static struct i2c_driver sc200ai_i2c_driver = {
        .driver = {
                .name = SC200AI_NAME,
                .pm = &sc200ai_pm_ops,
                .of_match_table = of_match_ptr(sc200ai_of_match),
        },
        .probe          = &sc200ai_probe,
        .remove         = &sc200ai_remove,
        .id_table       = sc200ai_match_id,
};

static int __init sensor_mod_init(void)
{
        return i2c_add_driver(&sc200ai_i2c_driver);
}

static void __exit sensor_mod_exit(void)
{
        i2c_del_driver(&sc200ai_i2c_driver);
}
```

#### 2.2.1.2 struct v4l2_subdev_ops

##### Description

`struct v4l2_subdev_ops` is the core structure of the **sub-device (Sub-device) operation set** in the Linux V4L2 framework, defining the functional interfaces of various hardware modules (such as Sensor, ISP, decoders).

##### Definition

```
struct v4l2_subdev_ops {
    const struct v4l2_subdev_core_ops *core;
    const struct v4l2_subdev_video_ops *video;
    const struct v4l2_subdev_pad_ops *pad;
    ...
};
```

##### Key Members

| Member Type | Structure | Function Description | Applicable Hardware |
| --- | --- | --- | --- |
| **Core Operations** | `const struct v4l2_subdev_core_ops *core` | Basic control (power/interrupt/logs, etc.) | All sub-devices |
| **Video Stream Control** | `const struct v4l2_subdev_video_ops *video` | Video stream parameters (frame rate/format/HDR, etc.) | Sensor/ISP/Encoders |
| **Panel Control** | `const struct v4l2_subdev_pad_ops *pad` | Pixel format/resolution/crop settings (**Key**) | Sensor/ISP/MIPI-CSI receivers |

**Example**

```
static const struct v4l2_subdev_ops sc200ai_subdev_ops = {
    .core   = &sc200ai_core_ops,
    .video  = &sc200ai_video_ops,
    .pad    = &sc200ai_pad_ops,
};
```

#### 2.2.1.3 struct v4l2_subdev_core_ops

**Description**

A set of operations used to implement core control functions in V4L2 sub-device drivers.

**Definition**

```
struct v4l2_subdev_core_ops {
    ...
        long (*ioctl)(struct v4l2_subdev *sd, unsigned int cmd, void *arg);
#ifdef CONFIG_COMPAT
        long (*compat_ioctl32)(struct v4l2_subdev *sd, unsigned int cmd,
                               unsigned long arg);
#endif
        int (*s_power)(struct v4l2_subdev *sd, int on);
    ...
};
```

**Key Members**

| Member Function | Function Prototype | Function Description | Typical Implementation in RK Sensor Driver |
| --- | --- | --- | --- |
| **`s_power`** | `int (*)(struct v4l2_subdev *sd, int on)` | **Control power state**: Turn on/off device power supply | Control the power sequence of PWDN/RESET pins, manage PMIC power timing (Power on: `on=1`, Power off: `on=0`) |
| **`compat_ioctl32`** | `long (*compat_ioctl32)(struct v4l2_subdev *sd, unsigned int cmd, unsigned long arg)` | Handle **private custom commands** (non-standard V4L2 commands) | When a 32-bit application uses a 64-bit kernel, implement private functions, **to solve the problem of data passed from user space/passed to user space** |
| **`ioctl`** | `int (*)(struct v4l2_subdev *sd, unsigned int cmd, void *arg)` | Handle **private custom commands** (non-standard V4L2 commands) | Implement unique features, respond to system ioctl calls |

**Example**

```
static const struct v4l2_subdev_core_ops sc200ai_core_ops = {
        .s_power = sc200ai_s_power,
        .ioctl = sc200ai_ioctl,
#ifdef CONFIG_COMPAT
        .compat_ioctl32 = sc200ai_compat_ioctl32,
#endif
};
```

Currently, the following private ioctl commands are used to implement module information queries and OTP information query settings.

| Private ioctl | Description |
| --- | --- |
| RKMODULE_GET_MODULE_INFO | Obtain module information |
| PREISP_CMD_SET_HDRAE_EXP | HDR exposure settings |
| RKMODULE_SET_HDR_CFG | Set HDR mode, which can switch between normal (linear) and HDR (high dynamic) modes, requiring driver adaptation to normal and HDR 2 sets of configuration information |
| RKMODULE_GET_HDR_CFG | Get the current HDR mode |
| RKMODULE_SET_CONVERSION_GAIN | Set the conversion gain in linear mode, such as imx347, os04a10 Sensors with conversion gain functionality, if the current Sensor does not support conversion gain, it can be left unimplemented. |

#### 2.2.1.4 struct v4l2_subdev_video_ops

**Description**

Video device control interface.

**Definition**

```
struct v4l2_subdev_video_ops {
        ...
        int (*s_stream)(struct v4l2_subdev *sd, int enable);
        int (*g_frame_interval)(struct v4l2_subdev *sd,
                                struct v4l2_subdev_frame_interval *interval);
        int (*s_frame_interval)(struct v4l2_subdev *sd,
                                struct v4l2_subdev_frame_interval *interval);
        ...

};
```

**Key Members**

| Member Name | Description |
| --- | --- |
| `.g_frame_interval` | Callback function for VIDIOC_SUBDEV_G_FRAME_INTERVAL ioctl |
| `.s_stream` | Key interface to notify the driver to start or stop video stream |
| `.s_frame_interval` | Core callback function for dynamically modifying the **Sensor's output frame rate** (frame interval), which is central to adjusting the frame rate |

**Example**

```
static const struct v4l2_subdev_video_ops sc200ai_video_ops = {
        .s_stream = sc200ai_s_stream,
        .g_frame_interval = sc200ai_g_frame_interval,
        .s_frame_interval = sc200ai_s_frame_interval,
};
```

#### 2.2.1.5 struct v4l2_subdev_pad_ops

##### Description

This structure defines a set of callback functions for operations such as **pixel format negotiation, cropping, scaling, and frame layout** on sub-devices; in platforms like RK, it is the key interface for configuring the image processing pipeline (Sensor → ISP → Memory).

##### Definition

```
struct v4l2_subdev_pad_ops {
        ...
        int (*enum_mbus_code)(struct v4l2_subdev *sd,
                              struct v4l2_subdev_pad_config *cfg,
                              struct v4l2_subdev_mbus_code_enum *code);
        int (*enum_frame_size)(struct v4l2_subdev *sd,
                               struct v4l2_subdev_pad_config *cfg,
                               struct v4l2_subdev_frame_size_enum *fse);
        int (*enum_frame_interval)(struct v4l2_subdev *sd,
                                   struct v4l2_subdev_pad_config *cfg,
                                   struct v4l2_subdev_frame_interval_enum *fie);
        int (*get_fmt)(struct v4l2_subdev *sd,
                       struct v4l2_subdev_pad_config *cfg,
                       struct v4l2_subdev_format *format);
        int (*set_fmt)(struct v4l2_subdev *sd,
                       struct v4l2_subdev_pad_config *cfg,
                       struct v4l2_subdev_format *format);
        int (*get_selection)(struct v4l2_subdev *sd,
                             struct v4l2_subdev_pad_config *cfg,
                             struct v4l2_subdev_selection *sel);
        int (*set_selection)(struct v4l2_subdev *sd,
                             struct v4l2_subdev_pad_config *cfg,
                             struct v4l2_subdev_selection *sel);
        int (*get_edid)(struct v4l2_subdev *sd, struct v4l2_edid *edid);
        int (*set_edid)(struct v4l2_subdev *sd, struct v4l2_edid *edid);
        int (*dv_timings_cap)(struct v4l2_subdev *sd,
                              struct v4l2_dv_timings_cap *cap);
        int (*enum_dv_timings)(struct v4l2_subdev *sd,
                               struct v4l2_enum_dv_timings *timings);
        ...

        int (*get_mbus_config)(struct v4l2_subdev *sd, unsigned int pad,
                               struct v4l2_mbus_config *config);
        int (*set_mbus_config)(struct v4l2_subdev *sd, unsigned int pad,
```

```
                                struct v4l2_mbus_config *config);
};
```

### Key Members

| Member Function | Function Description | Typical Application on Rockchip Platform |
| --- | --- | --- |
| **`enum_mbus_code`** | Enumerate supported media bus formats (pixel formats) | Report the RAW/YUV formats supported by the Sensor (e.g., `MEDIA_BUS_FMT_SRGGB10_1X10`) |
| **`get_fmt`** | Get the currently set format (resolution + pixel format) | Query the current output configuration of the Sensor (e.g., `1920x1080@SRGGB10`) |
| **`set_fmt`** | **Set the format** (resolution + pixel format), the **most important** function in the driver | Configure the Sensor output resolution (e.g., 1920x1080) and format (e.g., NV12), affecting MIPI data stream |
| **`enum_frame_size`** | Enumerate supported resolutions for a specified pixel format | List available resolutions for a specific format (e.g., UYVY) (e.g., 640x480, 1280x720) |
| **`enum_frame_interval`** | Enumerate supported frame rates for a specified format and resolution | Report supported resolutions, frame rates, etc., in the current mode |
| **`get_selection`** | Get the cropping/scaling area (read/write area, output size) | Implement digital zoom or set a small resolution area for low-power mode |
| **`set_selection`** | Set the cropping/scaling area | Use a 1080p area in preview mode |
| **`get_edid`** / **`set_edid`** | Handle EDID (Extended Display Identification Data) | Dedicated to HDMI receiver chips, usually ignored by Sensor drivers |
| **`dv_timings_cap`** | Get supported digital video timing standards | Dedicated to HDMITX/RX chips |
| **`enum_dv_timings`** | Enumerate supported digital video timings | Dedicated to HDMITX/RX chips |
| **`get_mbus_config`** | Get media bus configuration (e.g., MIPI D-PHY parameters) | Report:<br>• Lane count: `V4L2_MBUS_CSI2_4LANE`<br>• Clock mode: `V4L2_MBUS_CSI2_CONTINUOUS_CLOCK` |
| **`set_mbus_config`** | Set media bus configuration | Dynamically adjust MIPI parameters (e.g., switch between 2-lane/4-lane modes) |

### Example

```
static const struct v4l2_subdev_pad_ops sc200ai_pad_ops = {
        .enum_mbus_code = sc200ai_enum_mbus_code,
        .enum_frame_size = sc200ai_enum_frame_sizes,
        .enum_frame_interval = sc200ai_enum_frame_interval,
        .get_fmt = sc200ai_get_fmt,
        .set_fmt = sc200ai_set_fmt,
        .get_mbus_config = sc200ai_g_mbus_config,
};
```

#### 2.2.1.6 struct v4l2_ctrl_ops

##### Description

`struct v4l2_ctrl_ops` is the **core operations set** of the **V4L2 control framework**, used to implement dynamic control of sensor parameters (such as exposure, gain, white balance, etc.). In the development of Sensor drivers on the Rockchip platform, it provides a standardized interface to handle configuration requests from the application layer for sensor parameters, serving as a key hub connecting user space controls with hardware register operations.

##### Definition

```
struct v4l2_ctrl_ops {
        ...
        int (*s_ctrl)(struct v4l2_ctrl *ctrl);
};
```

##### Key Members

| Member Function | Function Description | Typical Implementation on Rockchip |
| --- | --- | --- |
| **`s_ctrl`** | **Set control value** (most important function) | Write the value set in user space to hardware registers (e.g., setting exposure time/gain) |

##### Example

```
static int sc200ai_set_ctrl(struct v4l2_ctrl *ctrl)
{
        struct sc200ai *sc200ai = container_of(ctrl->handler,
                                               struct sc200ai, ctrl_handler);
        struct i2c_client *client = sc200ai->client;

        ...

        if (!pm_runtime_get_if_in_use(&client->dev))
                return 0;

        switch (ctrl->id) {
        case V4L2_CID_EXPOSURE:
                dev_dbg(&client->dev, "set exposure value 0x%x\n", ctrl->val);
```

```
                ...
                break;
        case V4L2_CID_ANALOGUE_GAIN:
                dev_dbg(&client->dev, "set gain value 0x%x\n", ctrl->val);
                ...
                break;
        case V4L2_CID_VBLANK:
                dev_dbg(&client->dev, "set blank value 0x%x\n", ctrl->val);
                ...
                break;
        case V4L2_CID_TEST_PATTERN:
                ...
                break;
        case V4L2_CID_HFLIP:
                ...
                break;
        case V4L2_CID_VFLIP:
                ...
                break;
        default:
                dev_warn(&client->dev, "%s Unhandled id:0x%x, val:0x%x\n",
                         __func__, ctrl->id, ctrl->val);
                break;
        }

        pm_runtime_put(&client->dev);

        return ret;
}

static const struct v4l2_ctrl_ops sc200ai_ctrl_ops = {
        .s_ctrl = sc200ai_set_ctrl,
};
```

## 2.2.2 API Introduction

#### 2.2.2.1 xxxx_set_fmt

**Description**

Set the output format of the Sensor. This refers to a Sensor driver that supports multiple resolutions. By sending the resolution through set_fmt, the Sensor driver iterates to obtain the most suitable resolution. Switching between HDR and linear modes is not configured through this function, but through ioctl to implement private commands, achieving the switch. It should be noted that HDR and linear modes must have the same resolution to switch.

**Syntax**

```
static int xxxx_set_fmt(struct v4l2_subdev *sd,
                                                struct v4l2_subdev_pad_config *cfg,
                                                struct v4l2_subdev_format *fmt)
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *sd | Pointer to the v4l2 subdev structure | Input |
| *cfg | Pointer to the subdev pad information structure | Input |
| *fmt | Pointer to the Pad-level media bus format structure | Input |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

**Notes**

In kernel version 6.1, `struct v4l2_subdev_pad_config *cfg` has changed to `struct v4l2_subdev_state *sd_state`.

#### 2.2.2.2 xxxx_get_fmt

**Description**

Obtain the output format of the Sensor, which is the currently used configuration. As described earlier in the struct xxx_mode, multiple configurations with different resolutions may be set, and the driver will select one at initialization or switch after set_fmt. The current format can be confirmed through get_fmt. For chips before RK3588, the vc channel information currently used by the device is also uploaded through the reserved parameter fmt->reserved[0], which means this parameter can be used to specify the device's vc channel, otherwise, it will default. For RK3588 and later chips, the ioctl implementation RKMODULE_GET_CHANNEL_INFO is used to obtain channel information.

**Syntax**

```
static int xxxx_get_fmt(struct v4l2_subdev *sd,
                                                struct v4l2_subdev_pad_config *cfg,
                                                struct v4l2_subdev_format *fmt)
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *sd | Pointer to the v4l2 subdev structure | Input |
| *cfg | Pointer to the subdev pad information structure | Input |
| *fmt | Pointer to the Pad-level media bus format structure | Output |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

**Notes**

In kernel version 6.1, `struct v4l2_subdev_pad_config *cfg` has changed to `struct v4l2_subdev_state *sd_state`.

#### 2.2.2.3 xxxx_enum_mbus_code

**Description**

Enumerate Sensor output bus format. In the driver, a static structure variable is defined based on the struct xxx_mode, which is filled with the various resolutions supported by the Sensor driver. This function can return the enumeration value based on the bus format filled in the structure variable.

**Syntax**

```
static int xxxx_enum_mbus_code(struct v4l2_subdev *sd,
                                                    struct v4l2_subdev_pad_config
*cfg,
                                                    struct
v4l2_subdev_mbus_code_enum *code)
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *sd | Pointer to the v4l2 subdev structure | Input |
| *cfg | Pointer to the subdev pad information structure | Input |
| *code | Pointer to the media bus format enumeration structure | Output |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

**Notes**

In kernel version 6.1, `struct v4l2_subdev_pad_config *cfg` has been changed to `struct v4l2_subdev_state *sd_state`.

#### 2.2.2.4 xxxx_enum_frame_sizes

**Description**

Enumerates the output sizes of the Sensor. In the driver, a static structure variable is defined based on the struct xxx_mode, which is filled with the various resolutions supported by the Sensor driver. This function can return an enumeration value based on the resolutions filled in the structure variable.

**Syntax**

```
static int xxxx_enum_frame_sizes(struct v4l2_subdev *sd,
                                 struct v4l2_subdev_pad_config
*cfg,
                                 struct
v4l2_subdev_frame_size_enum *fse)
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *sd | Pointer to the v4l2 subdev structure | Input |
| *cfg | Pointer to the subdev pad information structure | Input |
| *fse | Pointer to the media bus frame size structure | Output |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

**Notes**

In kernel version 6.1, `struct v4l2_subdev_pad_config *cfg` has been changed to `struct v4l2_subdev_state *sd_state`.

#### 2.2.2.5 xxxx_g_frame_interval

##### Description

Obtain the frame interval output from the Sensor. The driver will define a static structure variable according to the struct xxx_mode structure, where the configured frame rate is the maximum frame rate. The driver reduces the frame rate by increasing the value of vblank. If the actual frame rate is needed, the value returned to the application needs to be converted by calculating the default vblank and the current vblank value to obtain the latest frame interval, which is uploaded to the application.

##### Syntax

```
static int xxxx_g_frame_interval(struct v4l2_subdev *sd,
                                                                  struct
v4l2_subdev_frame_interval *fi)
```

##### Parameters

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *sd | Pointer to the v4l2 subdev structure | Input |
| *fi | Pointer to the pad-level frame rate structure | Output |

##### Return Values

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 2.2.2.6 xxxx_s_stream

##### Description

Set stream input and output.

The function typically implements the start_stream/stop_stream functions based on the input parameter `on`. Inside `start_stream`, it initializes the configuration of the register array, initializes the exposure parameter configuration, and starts the data stream. Inside `stop_stream`, it implements the register configuration for turning off the stream.

##### Syntax

```
static int xxxx_s_stream(struct v4l2_subdev *sd, int on)
```

##### Parameters

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *sd | Pointer to v4l2 subdev structure | Input |
| on | 1: Start stream output; 0: Stop stream output | Input |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 2.2.2.7 xxxx_runtime_resume

**Description**

The callback function when the Sensor is powered on.

The main operation in this function is to power on the Sensor. Placing the power-on operation in this function facilitates the use of pm_runtime_get_sync by other parts of the Sensor driver or upper-level drivers to power on the Sensor.

**Syntax**

```
static int xxxx_runtime_resume(struct device *dev)
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Pointer to device struct | Input |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 2.2.2.8 xxxx_runtime_suspend

**Description**

Callback function when the Sensor is powered down.

The function performs the power-down operation of the Sensor, facilitating other parts of the Sensor driver or upper-layer drivers to use pm_runtime_put to power down.

**Syntax**

```
static int xxxx_runtime_suspend(struct device *dev)
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Pointer to device structure | Input |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 2.2.2.9 xxxx_set_ctrl

**Description**

Set the values of various v4l2 controls.

When registering v4l2 control commands, this callback function is invoked. When the application sets control parameters, this callback function is used to configure the Sensor.

For the v4l2 control commands that need to be implemented, refer to 3.2.1.6 for the implementation based on the explanation of struct v4l2_ctrl_ops.

**Syntax**

```
static int xxxx_set_ctrl(struct v4l2_ctrl *ctrl)
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *ctrl | Pointer to v4l2_ctrl structure | Input |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 2.2.2.10 xxx_enum_frame_interval

**Description**

Enumerates the frame interval parameters supported by the Sensor.

In the driver, a static structure variable is defined based on the struct xxx_mode structure, which can enumerate the frame intervals supported under several resolutions. This is generally not widely used, and frame rate adjustments are still made by adjusting vblank.

**Syntax**

```
static int xxxx_enum_frame_interval(struct v4l2_subdev *sd,
                                                                    struct
v4l2_subdev_pad_config *cfg,
                                                                    struct
v4l2_subdev_frame_interval_enum *fie)
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *sd | Sub-device instance | Input |
| *cfg | Pad configuration parameters | Input |
| *fie | Frame interval parameters | Output |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

**Notes**

In kernel version 6.1, `struct v4l2_subdev_pad_config *cfg` was changed to `struct v4l2_subdev_state *sd_state`.

#### 2.2.2.11 xxxx_g_mbus_config

**Description**

Retrieve the supported bus configurations. Bus types include MIPI/LVDS and others, with interfaces such as BT601/BT656/BT1120. For MIPI, there are DPHY/CPHY protocols. The controller uses this interface to obtain the bus parameters used by the current Sensor to confirm the controller's acquisition method.

For example, when using MIPI, if the Sensor supports multiple MIPI transmission modes, parameters can be uploaded based on the MIPI mode currently used by the Sensor.

#### Syntax

```
static int xxxx_g_mbus_config(struct v4l2_subdev *sd,
                                                    struct v4l2_mbus_config
*config)
```

#### Parameters

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *config | Bus configuration parameters | Output |

#### Return Values

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

### 2.2.2.12 xxxx_get_selection

#### Description

Configure the cropping parameters. The width of the ISP input must be aligned to 16, and the height must be aligned to 8. For resolutions from the Sensor that do not meet the alignment or are not standard resolutions, this function can be used to crop the resolution input to the ISP.

#### Syntax

```
static int xxxx_get_selection(struct v4l2_subdev *sd,
                                                  struct v4l2_subdev_pad_config *cfg,
                                                  struct v4l2_subdev_selection *sel)
```

#### Parameters

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *sd | Sub-device instance | Input |
| *cfg | Pad configuration parameters | Input |
| *sel | Cropping parameters | Output |

#### Return Values

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

**Notes**

In kernel version 6.1, `struct v4l2_subdev_pad_config *cfg` has been changed to `struct v4l2_subdev_state *sd_state`.

#### 2.2.2.13 xxxx_ioctl

**Description**

Implementation of private ioctl commands.

**Syntax**

```
static long xxxx_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *sd | Sub-device instance | Input |
| cmd | Private command value | Input |
| *arg | Argument passed | Input/Output |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

## 2.3 Driver Porting Steps

### 2.3.1 Implementing Standard I2C Sub-device Driver

1. Implement the following members according to the struct i2c_driver
   description:RKMODULE_SET_QUICK_STREAM

   struct driver.name

   struct driver.pm

struct driver.of_match_table probe function

remove function. (Note the difference in return values between kernel 5.10 and 6.1 versions)

2. Detailed description of the probe function implementation:

- Acquisition of CIS device resources, mainly parsing resources defined in the DTS file.
  - RK proprietary resource definition, naming convention as follows `rockchip,camera-module-xxx`, this part of the resources will be uploaded by the driver to the user space camera_engine to determine the matching of IQ effect parameters;
  - CIS device resource definition, RK related reference drivers generally include the following items:

| Member Name | Description |
| --- | --- |
| CIS Device Working Clock | External independent crystal solution does not need to be acquired, RK reference design generally uses AP output clock, this solution requires acquisition, generally named xvclk. |
| CIS Device Control GPIO | For example: Reset pin, Powerdown pin |
| CIS Device Control Power | According to the actual hardware design, obtain matching software power control resources, such as gpio, regulator |

  - CIS device ID number check, after obtaining the necessary resources through the above steps, it is recommended that the driver reads the device ID number to check the accuracy of the hardware, of course, this step is not a necessary step.
  - Initialization of CIS v4l2 device and media entity;

    v4l2 sub-device: v4l2_i2c_subdev_init, RK CIS driver requires subdev to have its own device node for user space rk_aiq access, through this device node to achieve exposure control;

    media entity: media_entity_init

### 2.3.2 Implementing v4l2 Sub-device Drivers

Mainly implement the following 3 members:

```
struct v4l2_subdev_core_ops
struct v4l2_subdev_video_ops
struct v4l2_subdev_pad_ops
```

1. Refer to the `struct v4l2_subdev_core_ops` documentation to implement its callback functions, mainly implementing the following callbacks:

- `.s_power` :

  s_power implements the power on and off operations for the Sensor. For some Sensors with a long array of registers, it is possible to summarize the common parts of the registers, which are executed after powering on, while s_stream only configures the registers that are necessary for different resolution configurations, speeding up the switching speed.

- **.compat_ioctl32**:
- **.ioctl**:

  ioctl mainly implements the RK proprietary control commands, involving:

| Member Name | Description |
| --- | --- |
| RKMODULE_GET_MODULE_INFO | Module information (module name, etc.) defined in the DTS file, which is uploaded to the camera_engine through this command for device matching |
| RKMODULE_AWB_CFG | When the module OTP information is enabled, the camera_engine passes the typical module AWB calibration values through this command, and the CIS driver is responsible for comparing them with the current module AWB calibration values and generating R/B Gain values to set in the CIS MWB module; |
| RKMODULE_LSC_CFG | When the module OTP information is enabled, the camera_engine controls the LSC calibration values to take effect through this command; |
| PREISP_CMD_SET_HDRAE_EXP | HDR exposure settings |
| RKMODULE_SET_HDR_CFG | Sets the HDR mode,supports switching between Normal and HDR modes, requiring the driver to adapt to HDR and normal 2 sets of configuration information |
| RKMODULE_GET_HDR_CFG | Obtains the current HDR mode |
| RKMODULE_SET_CONVERSION_GAIN | Sets the conversion gain in linear mode. For example, Sensors like imx347 and os04a10 have the function of conversion gain. High conversion gain can obtain better signal-to-noise ratio under low light conditions. If the Sensor does not support conversion gain, it can be left unimplemented. |
| RKMODULE_SET_QUICK_STREAM | Configures the Sensor's stream on/off registers, used for abnormal reset; supports multi-camera synchronization, used to uniformly start the data stream, detailed reference Multi-camera Synchronization Mechanism; quick start function; AOV function; |
| RKMODULE_GET_CHANNEL_INFO | Obtains channel information. By default, vicap's id0~id3 correspond to vc0~vc3. If there are special requirements, this command can be used to configure channel information, such as SPD/EBD data collection. RK3588 and earlier chips still obtain channel information through get_fmt. |
| RKMODULE_GET_SYNC_MODE, RKMODULE_SET_SYNC_MODEpixel_rate | Synchronization mode acquisition and configuration, detailed reference to multi-camera synchronization |

2. Refer to the `struct v4l2_subdev_core_ops` documentation to implement its callback functions.

   Mainly implement the following callbacks:

| Member Name | Description |
| --- | --- |
| .s_stream | The function to turn on and off the data stream. For MIPI CLK in continuous mode, the data stream must be turned on within this callback function. If the data stream is turned on prematurely, the MIPI LP state will not be recognized. |
| .g_frame_interval | Obtains the frame interval parameter (frame rate). |
| .g_mbus_config | Obtains the bus configuration. For MIPI interfaces, if the sensor driver supports different lane number configurations or supports HDR, the current sensor working mode's MIPI configuration is returned through this interface. |

3. Refer to the `struct v4l2_subdev_pad_ops` documentation to implement its callback functions.

   Mainly implement the following callback functions:

| Member Name | Description |
| --- | --- |
| .enum_mbus_code | Enumerates the data formats supported by the current CIS driver. For implementation, refer to xxxx_enum_mbus_code |
| .enum_frame_size | Enumerates the resolutions supported by the current CIS driver. For implementation, refer to xxxx_enum_frame_sizes |
| .get_fmt | The RKISP driver obtains the data format output by the CIS through this callback. It must be implemented. For Bayer raw Sensor, SoC yuv Sensor, BW raw Sensor output data types, refer to MEDIA_BUS_FMT List. For field output methods, refer to struct v4l2_mbus_framefmt definition; implementation refers to xxxx_get_fmt |
| .set_fmt | Sets the data format and resolution output by the CIS driver. It must be implemented. Implementation refers to xxxx_set_fmt |
| .enum_frame_interval | Enumerates the frame intervals supported by the sensor, including resolution. Implementation refers to xxx_enum_frame_interval |
| .get_selection | Configures the cropping parameters. The width required for ISP input is 16-aligned, and the height is 8-aligned. Implementation refers to xxxx_get_selection |

4. Refer to the `struct v4l2_ctrl_ops` documentation to implement.

   Mainly implement the following callback functions:

| Member Name | Description |
| --- | --- |
| .s_ctrl | The RKISP driver and camera_engine achieve CIS exposure control by setting different commands; |

The following IDs belong to the information acquisition category, and these parts are implemented according to the `standard integer menu controls` method:

| Member Name | Description |
| --- | --- |
| V4L2_CID_LINK_FREQ | Refers to the standard definition in CIS Driver V4L2-controls List. Currently, the RKISP driver obtains the MIPI bus frequency based on this command; |
| V4L2_CID_PIXEL_RATE | For MIPI bus: pixel_rate = link_freq * 2 * nr_of_lanes / bits_per_sample |
| V4L2_CID_HBLANK | Refers to the standard definition in CIS Driver V4L2-controls List |
| V4L2_CID_VBLANK | Refers to the standard definition in CIS Driver V4L2-controls List |

The RK camera_engine obtains necessary information through the above commands to calculate exposure, involving the following formulas:

| Formula |
| --- |
| line_time = HTS / PIXEL_RATE; |
| PIXEL_RATE = HTS * VTS * FPS |
| HTS = sensor_width_out + HBLANK; |
| VTS = sensor_height_out + VBLANK; |

The following IDs belong to the control category. The RK camera_engine controls the CIS through these commands:

| Member Name | Description |
| --- | --- |
| V4L2_CID_VBLANK | Adjusts VBLANK to adjust the frame rate and Exposure time max; |
| V4L2_CID_EXPOSURE | Sets the exposure time, unit: exposure lines |
| V4L2_CID_ANALOGUE_GAIN | Sets the exposure gain, actually total gain = analog gain*digital gain; unit: gain register value |
| V4L2_CID_HFLIP | Sets horizontal mirroring. The ISP does not have mirroring functionality. If mirroring is needed, use the Sensor's mirroring mechanism |
| V4L2_CID_VFLIP | Sets vertical mirroring. The ISP does not have mirroring functionality. If mirroring is needed, use the Sensor's mirroring mechanism |
| V4L2_CID_TEST_PATTERN | Implements the test pattern, not mandatory, can be used for debugging, implement as needed |

5. HDR Driver Implementation Considerations:

- Linear mode exposure and gain are implemented through standard v4l2 commands. When switching to HDR mode, parameters are passed through the ioctl command PREISP_CMD_SET_HDRAE_EXP. The standard v4l2 commands V4L2_CID_EXPOSURE and V4L2_CID_ANALOGUE_GAIN are not used in HDR mode. When implementing PREISP_CMD_SET_HDRAE_EXP, it is necessary to cache the parameters. After writing the initialization array in start_stream, the initial exposure needs to be rewritten into the Sensor, covering the default values in the initialization array. This ensures that the brightness of the first frame output matches the exposure parameters, allowing for faster AE convergence. It is important to note that the initialization array should not configure registers to start the data stream, which should be configured before start_stream returns, also to ensure that the brightness of the first frame output matches the initial exposure parameters.
- Sensor and controller HDR/LINEAR interaction is through RKMODULE_SET_HDR_CFG/RKMODULE_GET_HDR_CFG. HDR Sensors must implement this to switch modes normally.

## 2.4 Modifying Makefile

Modify the `kernel/drivers/media/i2c/Makefile` file to add the compilation of the newly added Sensor.

Take SC200AI as an example:

```
......
obj-$(CONFIG_VIDEO_SC200AI) += sc200ai.o
......
```

Note:

Configurations should be added in alphabetical order according to the manufacturer and Sensor naming convention.

## 2.5 Kconfig Modifications

Modify the `kernel/drivers/media/i2c/Kconfig` file to add the configuration and description for the newly added Sensor.

Taking SC200AI as an example:

```
config VIDEO_SC200AI
        tristate "SmartSens SC200AI sensor support"
        depends on I2C && VIDEO_V4L2
        select MEDIA_CONTROLLER
        select VIDEO_V4L2_SUBDEV_API
        select V4L2_FWNODE
        help
          This is a Video4Linux2 sensor driver for the SmartSens
          SC200AI camera.
```

Note:

Configurations should be added in alphabetical order according to the manufacturer's name, i.e., Sensor naming.

# 3. Quick Start

## 3.1 Concepts

**Sensor Quick Boot Mode** refers to a mode supported by **image sensors (such as CMOS Sensors)** that allows for rapid power-up, initialization, and entry into the working state. The purpose is to **enable the Sensor to complete startup, initialization, and output valid image data in the shortest possible time**, thereby meeting the system's requirements for quick startup, low latency, and real-time performance.

In many embedded systems, consumer electronics, automotive electronics, or industrial vision applications, **the system requires the camera to output images "as soon as possible" after power-up**, such as:

- Immediate capture / quick snapshot
- Strict system startup time requirements (e.g., <100ms / 300ms to produce images)
- Waking up the camera from power-saving mode and quickly producing images
- Quick switching or synchronized startup in multi-camera systems
- High real-time requirements for video calls / surveillance, etc.

Traditional Sensor startup processes may take a relatively long time (for example, several hundred milliseconds or even more than 1 second), including:

1. Power-up
2. Internal PLL / clock stabilization
3. Register initialization
4. Analog circuit stabilization
5. Establishment of line and field timing
6. Output of the first frame of valid image

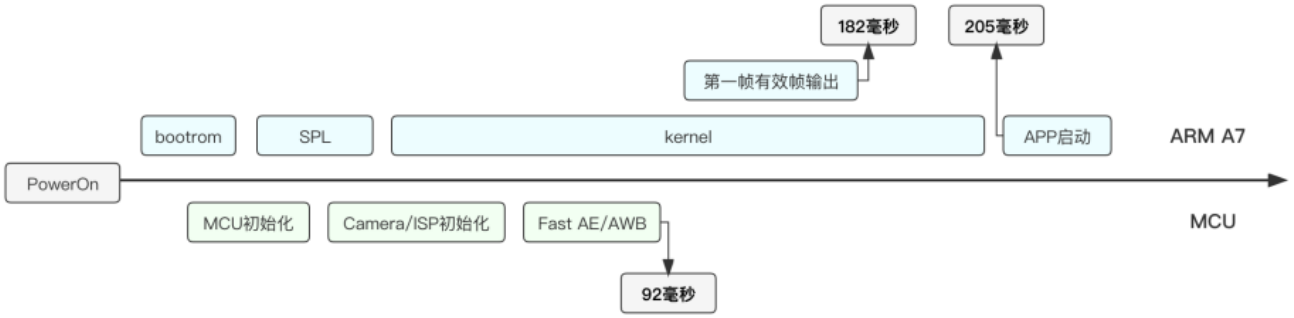
The Quick Boot Mode, by optimizing the internal startup timing of the sensor, skipping some unnecessary initialization steps, or pre-configuring key registers in advance, allows the Sensor to **enter a state where images can be output more quickly**.

**Comparison of Typical Startup Times for Different Startup Modes**

| Startup Mode | Typical Startup to Image Output Time | Applicable Scenarios |
|---|---|---|
| **Normal Boot** | 300ms ~ 1000ms | General applications with no strict requirements for startup time |
| **Fast Boot** | 10ms ~ 200ms | Quick snapshot, real-time preview, low-latency systems |
| **Low-power Wakeup** | 10ms ~ 50ms | Fast recovery from power-saving mode (better when combined with Fast Boot) |

RK RV1103, RV1103B/RV1106B, RV1126B, and other main control platforms have a high-performance MCU RISCV core built-in, transferring some functions of ISP AE AWB to the MCU, allowing it to assist ARM in initializing the Camera and ISP, achieving rapid convergence of image effects. From power-up to the startup of the LINUX kernel system, a certain amount of time is required. During this phase, the MCU parallelly achieves AE automatic exposure convergence and day/night mode judgment. When the kernel ISP driver is registered, the Sensor uses the exposure amount calculated by the MCU for exposure to produce images, and related image effect parameters are obtained from the MCU, thereby achieving the goal of quickly capturing the first frame of valid images, while the image effects are essentially stable and converged.

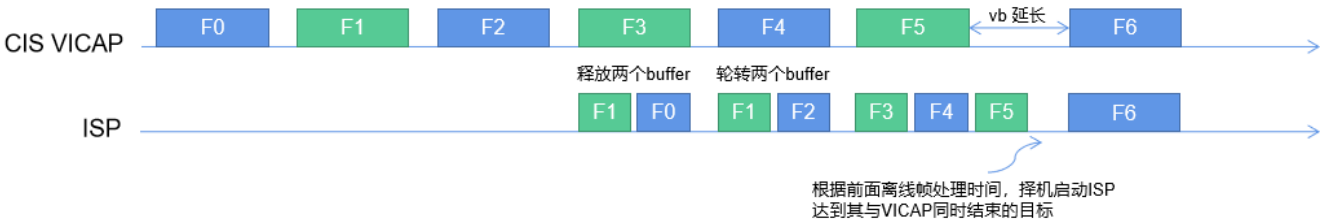
The basic process of quick startup is shown in the figure:




The process of MCU-assisted rapid convergence of image effects is shown in the following figure:




The specific process of ARM phase output is as follows:




## 3.2 Driver-Related

### 3.2.1 Kernel Drivers

1. Include the following two header files:

```
#include "../platform/rockchip/isp/rkisp_tb_helper.h"
#include "cam-tb-setup.h"
```

2. Add the `is_thunderboot` variable to the Sensor device structure to identify whether it is in quick boot mode.

```
struct sc200ai {
   ......
   bool            is_thunderboot;
   ......
};
```

In the probe interface, check if the quick boot related configuration is set: `CONFIG_VIDEO_ROCKCHIP_THUNDER_BOOT_ISP`

If the kernel is configured with this option, `is_thunderboot` will be `true`.

```
sc200ai->is_thunderboot = IS_ENABLED(CONFIG_VIDEO_ROCKCHIP_THUNDER_BOOT_ISP);
```

3. Start and stop stream interfaces

```
static int __sc200ai_start_stream(struct sc200ai *sc200ai)
{
   int ret = 0;
   ......
   if (!sc200ai->is_thunderboot) {   //Non-quick boot, write complete register configuration
      ret = sc200ai_write_array(sc200ai->client, sc200ai->cur_mode->reg_list);
      if (ret)
         return ret;
      ......
   }

   if (sc200ai->sync_mode == NO_SYNC_MODE) //Otherwise, configure to start streaming
      ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                               SC200AI_REG_VALUE_08BIT, SC200AI_MODE_STREAMING);
   return ret;
}

static int __sc200ai_stop_stream(struct sc200ai *sc200ai)
{
   int ret = 0;
   sc200ai->has_init_exp = false;

   if (sc200ai->is_thunderboot) {
     sc200ai->is_first_streamoff = true;
     pm_runtime_put(&sc200ai->client->dev);
   }
   ......

   ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                            SC200AI_REG_VALUE_08BIT, SC200AI_MODE_SW_STANDBY);
   return ret;
}
```

### 3.2.2 MCU Driver

Refer to Camera Driver Porting

## 3.3 MCU Quick Start Configuration Guide

### 3.3.1 MCU Driver Framework and Introduction

The following sections take the SmartSens SC450AI Sensor driver development as an example to illustrate the process of MCU Sensor driver development on the RK platform.

#### 3.3.1.1 Data Type Description

##### 3.3.1.1.1 struct rk_camera_ops

**Description**

The struct rk_camera_ops defines the operational interface for camera devices.

**Definition**

```
struct rk_camera_ops {
    ret_err_t (*init)(struct rk_camera_device *dev);  // Initialize camera device
    ret_err_t (*open)(struct rk_camera_device *dev, rt_uint16_t oflag);  // Open camera
device
    ret_err_t (*close)(struct rk_camera_device *dev);  // Close camera device
    ret_err_t (*control)(struct rk_camera_device *dev, int cmd, void *args);  // Control
camera device
};
```

**Members**

| Member Name | Description |
| --- | --- |
| `init` | Initialize device |
| `open` | Open device |
| `close` | Close device |
| `control` | Control device |

**Example**

```
struct rk_camera_ops rk_sc450ai_ops =
{
    rk_sc450ai_init,
    rk_sc450ai_open,
    NULL,
    rk_sc450ai_control
};
```

#### 3.3.1.1.2 struct xxx_mode

**Description**

Sensor supports information for various modes, and this structure can be expanded with variables as needed.

**Definition**

```
struct xxxx_mode
{
    struct v4l2_fract max_fps;
    uint16_t bus_fmt;
    uint16_t width;
    uint16_t height;
    uint16_t hts_def;
    uint16_t vts_def;
    uint16_t exp_def;
    uint32_t xvclk;
    uint32_t link_freq;
#ifndef REG_LIST_CONVERSION_DEBUG
    const uint8_t *reg_list;
#else
    const struct sc450ai_sensor_reg *reg_list;
#endif
    const int reg_list_size;
    uint8_t hdr_mode;
} __attribute__((__packed__));
```

**Members**

| Member Name | Description |
| --- | --- |
| .max_fps | Image frame rate, denominator/numerator = fps. |
| .bus_fmt | Sensor output format. |
| .width | Effective image width. Must match the current configured width of the sensor. |
| .height | Effective image height. Must match the current configured height of the sensor. |
| .hts_def | Default HTS, which is the effective image width + HBLANK. |
| .vts_def | Default VTS, which is the effective image height + VBLANK. |
| .exp_def | Default exposure time. |
| .xvclk | Sensor external clock frequency, i.e., MCLK. |
| .link_freq | MIPI_DPHY sampling frequency (in MHz), which is half of the MIPI rate. |
| *reg_list | Register list. |
| .reg_list_size | Size of the register array. |
| .hdr_mode | Sensor working mode, supports linear mode, two-frame HDR synthesis, and three-frame HDR synthesis. |

**Example**

```
static const struct sc450ai_mode supported_modes_4lane[] =
{
    {
        .bus_fmt = MEDIA_BUS_FMT_SBGGR10_1X10,
        .width = 1344,
        .height = 760,
        .max_fps = {
            .numerator = 10000,
            .denominator = 1200000,
        },
        .exp_def = 0x80,
        .hts_def = 0x03a8,
        .vts_def = 0x030c,
        .xvclk = 27000000,
        .link_freq = 360000000,
        .reg_list = g_sc450ai_1344x760_4lane_120fps_reg_table,
        .reg_list_size = ARRAY_SIZE(g_sc450ai_1344x760_4lane_120fps_reg_table),
        .hdr_mode = NO_HDR,
    },
    {/* */},
}
```

#### 3.3.1.1.3 Struct xxx_dev

### Description

The primary data structure for the Camera device, containing device configuration, status information, and operation function pointers.

Configured in the rk_xxx_init function.

### Definition

```
struct xxx_dev
{
    struct rk_camera_device parent;
    char name[RK_CAMERA_DEVICE_NAME_SIZE];
    uint8_t time_valid_delay;
    uint8_t gain_valid_delay;
    char i2c_name[RK_CAMERA_I2C_NAME_SIZE];
    struct rt_i2c_bus_device *i2c_bus;
    uint8_t i2c_addr;
    uint8_t flip;
    struct rt_mutex mutex_lock;
    struct rk_camera_exp_val init_exp;
    uint8_t modes_num;
    const struct sc450ai_mode *supported_modes;
    const struct sc450ai_mode *cur_mode;
    const struct sc450ai_mode *pdst_mode;
    struct sc450ai_mode dst_mode;
    bool has_init_exp;
    bool streaming;
};
```

### Members

| Member Name | Description |
| --- | --- |
| parent | Parent device of the Camera |
| name[RK_CAMERA_DEVICE_NAME_SIZE] | Camera name |
| time_valid_delay | Exposure time takes effect in N + time_valid_delay frames |
| gain_valid_delay | Exposure gain takes effect in N + gain_valid_delay frames |
| i2c_name[RK_CAMERA_I2C_NAME_SIZE] | I2C bus name |
| *i2c_bus | I2C bus device |
| i2c_addr | I2C address |
| flip | Flip/mirror setting |
| mutex_lock | Mutex lock |
| init_exp | Initial exposure value |
| modes_num | Number of initial configurations |
| *supported_modes | List of modes corresponding to the lane count (todo) |
| *cur_mode | Current mode parameters |
| *dst_mode | Destination mode parameters |
| has_init_exp | Records whether the initial exposure has been set |
| streaming | Records whether the device is in streaming state |

#### 3.3.1.2 API Description

##### 3.3.1.2.1 rk_xxx_control

**Description**

This function is a generic control interface used to apply/ISP to send various device control commands.

**Syntax**

```
ret_err_t rk_sc450ai_control(struct rk_camera_device *dev,
                             dt_cmd_t cmd,
                             void *args);
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| `*dev` | Camera device descriptor | Input |
| `cmd` | Command to control the Camera device | Input |
| `*args` | Command arguments | Input |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

**Example**

```
ret_err_t rk_sc450ai_control(struct rk_camera_device *dev,
                             dt_cmd_t cmd,
                             void *args)
{
    ...
    switch (cmd)
    {
    case RK_DEVICE_CTRL_DEVICE_INIT:
    {
        ret = rk_sc450ai_init(dev);
    }
    break;
    ...
    default:
        sc450ai_dbg("exit CMD %d\n", cmd);
        break;
    }

    return ret;
}
```

#### 3.3.1.2.2 rk_xxx_init

**Description**

The controller retrieves the bus parameters currently used by the Camera through this interface to confirm the controller's acquisition method.

**Syntax**

```
ret_err_t rk_xxx_init(struct rk_camera_device *dev);
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera device descriptor | Input |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 3.3.1.2.3 xxx_stream_on

**Description**

Configure the Camera initialization registers and determine whether to start the data stream early based on the RT_USING_CAM_STREAM_ON_LATE macro.

**Syntax**

```
static void xxx_stream_on(struct xxx_dev *dev);
```

**Parameter**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera device descriptor | Input |

**Return value**

| Return Values | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 3.3.1.2.4 xxx_stream_off

**Description**

Controls the camera to stop data streaming.

**Syntax**

```
static void xxx_stream_off(struct xxx_dev *dev);
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera device descriptor | Input |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 3.3.1.2.5 rk_xxx_get_expinf

**Description**

Obtaining Camera Exposure Information.

**Syntax**

```
static ret_err_t rk_xxx_get_expinf(struct xxx_dev *dev, struct rk_camera_exp_info *exp);
```

**Parameter**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera Device Descriptor | Input |
| *exp | Parameters Related to Camera Exposure Information | Output |

**Return value**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 3.3.1.2.6 rk_xxx_set_expval

**Description**

Set the exposure time and gain of the Camera.

**Syntax**

```
static ret_err_t rk_xxx_set_expval(struct xxx_dev *dev, struct rk_camera_exp_val *exp);
```

**Parameter**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera device descriptor | Input |
| *exp | Parameters related to camera exposure | Input |

**Return value**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 3.3.1.2.7 rk_xxx_set_vts

**Description**

Setting up Camera VTS.

**Syntax**

```
static ret_err_t rk_xxx_set_vts(struct xxx_dev *dev, uint32_t dst_vts);
```

**Parameter**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera device descriptor | Input |
| dst_vts | Target VTS value | Input |

**Return value**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 3.3.1.2.8 rk_xxx_get_input_fmt

**Description**

Obtains the camera output format, which is the currently used configuration. As described earlier in the struct xxx_mode, multiple configurations with different resolutions may be set. The driver selects one of them during initialization, or after switching with set_input_fmt, the current format can be confirmed through get_input_fmt.

**Syntax**

```
static ret_err_t rk_xxx_get_input_fmt(struct xxx_dev *dev, struct rk_camera_mbus_framefmt
*mbus_fmt);
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera device descriptor | Input |
| *mbus_fmt | Camera output format | Output |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 3.3.1.2.9 rk_xxx_set_input_fmt

**Description**

Set the camera output format and configure the controller's acquisition parameters.

**Syntax**

```
static ret_err_t rk_xxx_set_input_fmt(struct xxx_dev *dev, struct rk_camera_mbus_framefmt
*mbus_fmt);
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera device descriptor | Input |
| *mbus_fmt | Camera's output format | Input |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 3.3.1.2.10 rk_xxx_match_dst_config

**Description**

Based on the target configuration, iterate through all supported_modes to find the output format for the image with the target resolution and determine if there is frame rate reduction. If frame rate reduction is present, modify the target VTS.

**Syntax**

```
static ret_err_t rk_xxx_match_dst_config(struct xxx_dev *dev, struct rk_camera_dst_config
*dst_config);
```

**Parameters**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera device descriptor | Input |
| *dst_config | Output format of target config | Input |

**Return Values**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 3.3.1.2.11 rk_xxx_set_flip_mirror

**Description**

Set Camera flip or mirror.

**Syntax**

```
static ret_err_t rk_xxx_set_flip_mirror(struct xxx_dev *dev, uint32_t flip)
```

**Parameter**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera device descriptor | Input |
| flip | 0: No flip no mirror 1: Only flip 2: Only mirror 3: Flip and mirror | Input |

**Return value**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

#### 3.3.1.2.12 xxx_stream_on_late

#### Description

Set initial exposure, mirroring, flipping, and enable data stream for the Camera.

#### Syntax

```
static void xxx_stream_on_late(struct xxx_dev *dev);
```

#### Parameters

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera device descriptor | Input |

#### Return Values

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

##### 3.3.1.2.13 xxx_quick_stream

#### Description

This function is used to start or stop the Camera data stream.

#### Syntax

```
static void xxx_quick_stream(struct xxx_dev *dev, int on);
```

#### Parameter

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera device descriptor | Input |
| on | 1: Start stream output; 0: Stop stream output | Input |

#### Return value

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

##### 3.3.1.2.14 rk_xxx_check_sensor_id

#### Description

This function is used for the registration of the Sensor I2C device, and it is necessary to configure it according to the power-up sequence in the Sensor Datasheet.

**Syntax**

```
int rk_xxx_check_sensor_id(struct xxx_dev *dev)
```

**Parameter**

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| *dev | Camera device descriptor | Input |

**Return value**

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure |

## 3.3.2 Camera Driver Porting

When writing a new Sensor driver, it is possible to base it on a driver for a Sensor with similar specifications (`rt-thread/bsp/rockchip/camera/`) or modify a driver from the same manufacturer with a similar model.

### 3.3.2.1 Adding Compilation Configuration Options

In `rt-thread/bsp/rockchip/common/drivers/Kconfig`, make the following modifications.

```
@@ -506,6 +506,8 @@ menu "supported camera"
        bool "Enable GC2053"
    config RT_USING_F37
        bool "Enable F37"
    config RT_USING_SC230AI
        bool "Enable SC230AI"
+   config RT_USING_SC450AI
+       bool "Enable SC450AI"
    config RT_USING_SC850SL
        bool "Enable SC850SL"
```

### 3.3.2.2 Register Sequence Conversion

In the quick start mode, the MCU Sensor driver uses the I2C continuous write method to input the initialization configuration. The configuration provided by the original factory is generally in a non-continuous format, which needs to be converted into the continuous write format corresponding to the driver using a script.

#### 3.3.2.2.1 Sequence Conversion Steps

1. Refer to the format in `<sdk>/sysdrv/source/mcu/project/scripts/camera_reg_convert/reg.txt` to modify the initialization sequence.
2. Replace the initialization sequence in `<sdk>/sysdrv/source/mcu/project/scripts/camera_reg_convert/reg.txt`.
3. Use the following command to perform the conversion:

```
cd <sdk>/sysdrv/source/mcu/project/scripts/camera_reg_convert/
python2 reg_conver.py
```

4. Copy and replace the conversion results into the corresponding register sequence in the Sensor driver.

**Example**

```
/*
 * Xclk 27MHz
 * max_framerate 30fps
 * mipi_datarate per lane 720Mbps, 4lane
 * full resolution
 */
static const uint8_t g_sc450ai_2688x1520_4lane_30fps_reg_table[] =
{
    0x3, 0x01, 0x00, 0x00,
    0x3, 0x36, 0xe9, 0x80,
    0x3, 0x36, 0xf9, 0x80,
    0x3, 0x30, 0x1c, 0x78,
    0x3, 0x30, 0x1f, 0x01,
    0x3, 0x30, 0x2e, 0x00,
    0xa, 0x32, 0x08, 0x0a, 0x80, 0x05, 0xf0, 0x02, 0xee, 0x06, 0x18,
    0x4, 0x32, 0x14, 0x11, 0x11,
    0x8,
};
```

**Description**

Continuous write format description

Column 1 represents the number of I2C DATA bytes written at a time.

Columns 2-3 represent the I2C register address.

Columns 4-n represent the continuous I2C register values written.

The last line indicates the total number of I2C write operations.

**Precautions**

Do not enable the output flow register in the register sequence; enable it separately in the xxx_stream_on_late function.

#### 3.3.2.3 Configuring Basic Functions

Implement all functions within rk_camera_ops, including all controls within rk_xxx_control.

Implement the rk_camera_xxx_init function.

## 3.3.3 MCU Board-Level Configuration Guide

### 3.3.3.1 CIS (CMOS Image Sensor) Device Registration

#### 3.3.3.1.1 struct gpio_desc

**Definition**

```
struct gpio_desc
{
    uint32_t gpio_pin;
    struct GPIO_REG *gpio_group;
};
```

#### 3.3.3.1.2 camera_board_desc Structure Description

**Description**

Configuration information related to the sensor and board-level hardware

**Definition**

```
struct camera_board_desc
{
    struct gpio_desc rst_gpio;
    struct gpio_desc pwdn_gpio;
    struct gpio_desc pwren_gpio;
    eCLOCK_Name mclk_id;
    uint32_t mclk_out_gate_id;
    char isp_subdev_name[RK_CAMERA_DEVICE_NAME_SIZE];
    char i2c_bus[RK_CAMERA_I2C_NAME_SIZE];
    uint8_t i2c_addr;
    uint8_t time_valid_delay;
    uint8_t gain_valid_delay;
    uint8_t mode_id;
    uint32_t width;
    uint32_t height;
    uint32_t pixelcode;
    uint32_t maxfps;
    uint32_t lanes;
    uint32_t hdr_mode;
```

```
    uint32_t dst_width;
    uint32_t dst_height;
    uint32_t dst_pixelcode;
    uint32_t dst_maxfps;
    uint32_t dst_hdr_mode;
    bool rst_active;
    bool pwdn_active;
    bool pwren_active;
};
```

### Members

| Member Name | Description |
| --- | --- |
| rst_gpio | Hardware reset pin configuration, can be implemented according to the definition of gpio_desc. |
| pwdn_gpio | Power-down mode pin configuration, can be implemented according to the definition of gpio_desc. |
| pwren_gpio | Power-on pin configuration, can be implemented according to the definition of gpio_desc. |
| mclk_id | Select mclk0 or mclk1 by configuring 0 or 1. |
| isp_subdev_name | Whether the sensor is connected to ISP as the main camera. Main camera: RK_ISP_MAIN_SUBDEV_NAME Assist camera: RK_ISP_ASSIST_SUBDEV_NAME |
| i2c_bus | Select the Sensor I2C bus |
| i2c_addr | Configure the Sensor I2C address (seven-bit address) |
| time_valid_delay | The number of frames required for the exposure time to take effect |
| gain_valid_delay | The number of frames required for the exposure gain to take effect |
| mode_id | Select the sensor resolution, can be configured according to the resolution settings in camera.h |
| width | The width of the MCU small image |
| height | The height of the MCU small image |
| pixelcode | The image format of the MCU small image |
| maxfps | The maximum frame rate of all initialized configurations for the MCU small image |
| lanes | The number of Sensor lanes |
| hdr_mode | The HDR mode of the MCU small image (NO_HDR: 0 HDR_X2: 5 HDR_X3: 6 ) |
| dst_width | The width of the MCU large image |
| dst_height | The height of the MCU large image |
| dst_pixelcode | The image format of the MCU large image |
| dst_maxfps | The maximum frame rate of all initialized configurations for the MCU large image |
| dst_hdr_mode | The HDR mode of the MCU large image (NO_HDR: 0 HDR_X2: 5 HDR_X3: 6 ) |
| rst_active | The valid voltage level of the hardware reset pin |
| pwdn_active | The valid voltage level of the power-down mode pin |
| pwren_active | The valid voltage level of the power-on pin |

#### 3.3.3.1.3 `cif_hw_config` Function Description

data_lanes: Number of sensor lanes

Connection relationship between Sensor, CSI_DPHY, and CSI_HOST is shown in the table below

| Main Camera and Dphy Hardware Connection Relationship | dphy1.enable | dphy2.enable | csi_host_0.enable | csi_host_1.enable |
|---|---|---|---|---|
| lane0 <-> MIPI_CSI_RX_D0N/P | 1 | 0 | 1 | 0 |
| lane1 <-> MIPI_CSI_RX_D1N/P | 1 | 0 | 1 | 0 |
| lane2 <-> MIPI_CSI_RX_D2N/P | 0 | 1 | 0 | 1 |
| lane3 <-> MIPI_CSI_RX_D3N/P | 0 | 1 | 0 | 1 |

#### 3.3.3.1.4 Single Camera Registration

**Example**

```
const struct camera_board_desc camera_sc450ai =
{
    .i2c_addr = 0x30,
    .rst_gpio =
    {
        .gpio_pin = GPIO_PIN_A7,
        .gpio_group = GPIO4,
    },
    .pwdn_gpio =
    {
        .gpio_pin = GPIO_PIN_A2,
        .gpio_group = GPIO4,
    },
    .time_valid_delay = 2,
    .gain_valid_delay = 2,
    .width = 1344,
    .height = 760,
    .maxfps = 120,
    .lanes = 4,
    .pixelcode = MEDIA_BUS_FMT_SBGGR10_1X10,
    .hdr_mode = NO_HDR,
    .dst_width = 2688,
    .dst_height = 1520,
    .dst_maxfps = 60,
    .dst_pixelcode = MEDIA_BUS_FMT_SBGGR10_1X10,
    .dst_hdr_mode = NO_HDR,
    .isp_subdev_name = RK_ISP_MAIN_SUBDEV_NAME,
    .i2c_bus = "i2c3",
    .rst_active = 0,
    .pwdn_active = 1,
    .mclk_id = CLK_MIPI0_OUT2IO,
    .mclk_out_gate_id = CLK_MIPI0_OUT2IO_GATE,
};
```

```
void cif_hw_config(void)
{
```

```
    struct dphy_board_desc *csi2_dphy = (struct dphy_board_desc *)&csi2_dphy_board;
    struct csi2_board_desc *csi_host_0 = (struct csi2_board_desc *)&csi2_host0_board;
    struct vicap_board_desc *vicap = (struct vicap_board_desc *)&vicap_board;

    csi2_dphy->enable = true;
    csi2_dphy->csi2_dphy1.enable = true;
    csi2_dphy->csi2_dphy1.csi_host_idx = 0;
    csi2_dphy->csi2_dphy1.isp_subdev_name = RK_ISP_MAIN_SUBDEV_NAME;

    csi_host_0->enable = true;

    vicap->enable = true;
}
```

### 3.3.4 How to Add Board-Level Configuration

1. Implement the `camera_board_desc` structure and `cif_hw_config` function in `bsp/rockchip/xxx_mcu/board/xxx/board.c`. Refer to CIS Device Registration
2. In `bsp/rockchip/xxx_mcu/board/xxx/iomux.c`, add peripheral iomux configurations based on the hardware schematics.

# 4. New Feature Support

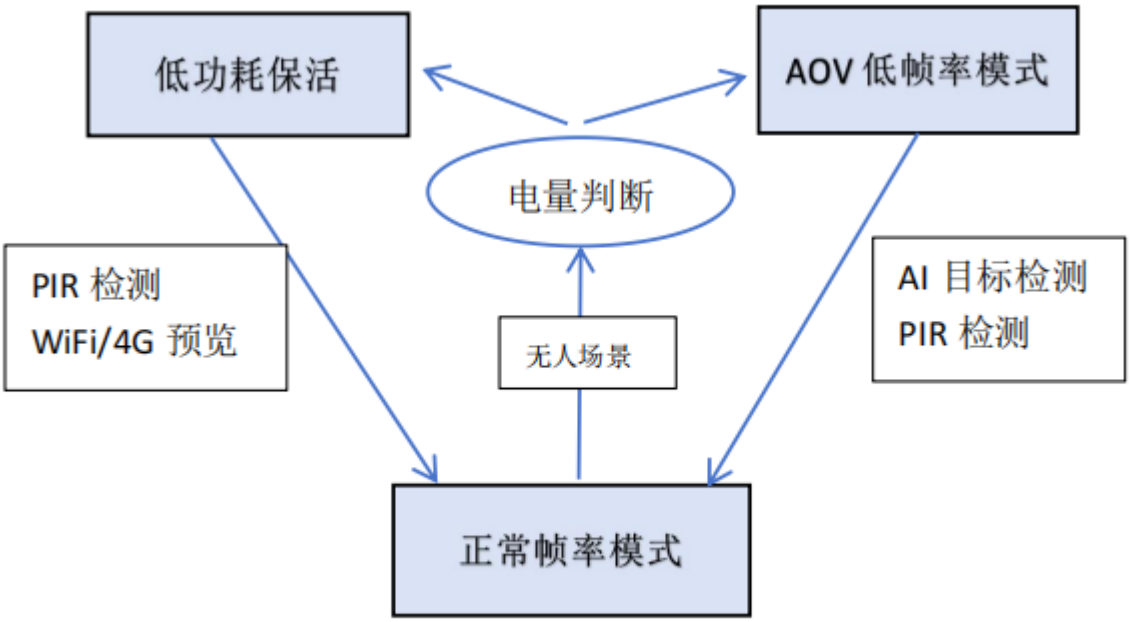
## 4.1 AOV

AOV (Always On Video) is a 24-hour low frame rate continuous recording feature based on sleep-wake functionality. It can switch back to normal frame rate mode upon events, such as AI detection, PIR triggering, WiFi wake-up, etc.

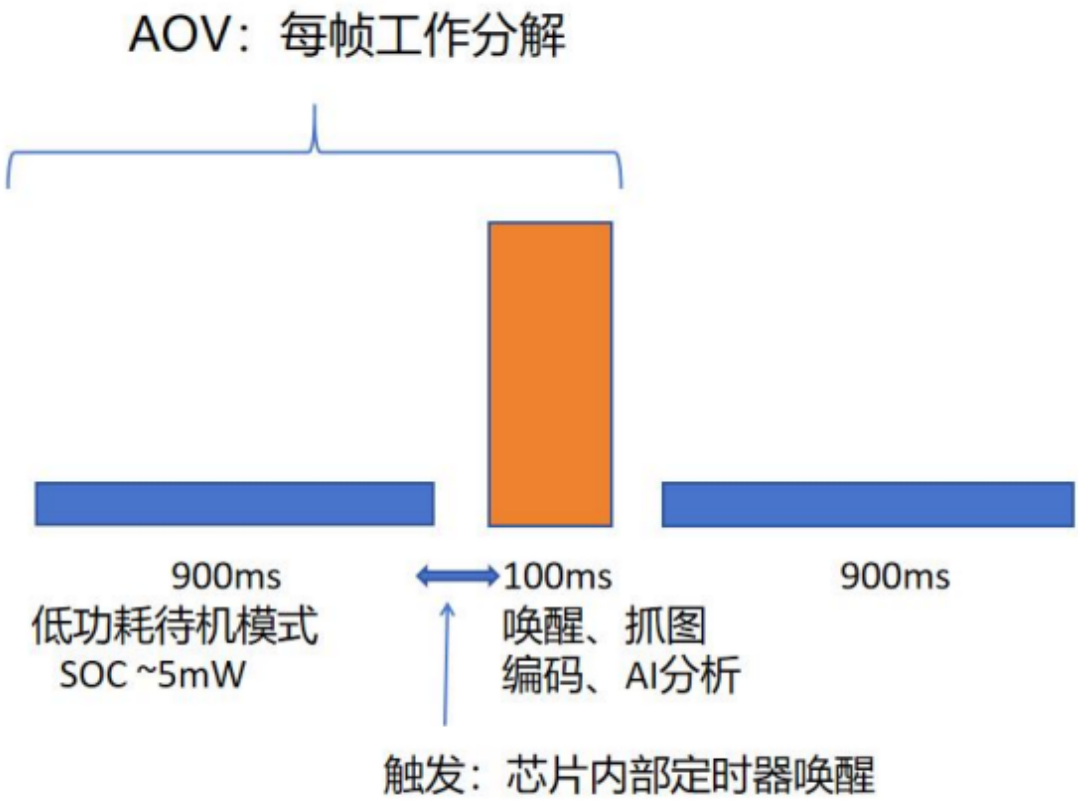
### 4.1.1 Scenario Description

**Three Scenarios**:

- Normal Frame Rate Mode: Enters during remote preview, PIR event, and human shape detection events. Requires recording and internet connectivity functions;
- AOV Low Frame Rate Mode: When idle and with sufficient battery, SoC records low frame rate primary stream video, WiFi/4G stays active;
- Low Power Consumption Keep-Alive Mode: When idle and with low battery, SoC powers down, WiFi/4G stays active.




### 4.1.2 Workflow




### 4.1.3 Concept Introduction

AOV mode involves the standby mode of the Sensor.

The standby mode refers to the Sensor being in sleep mode or reset mode to conserve power consumption.

- Sleep Mode

  The Sensor stops the image data stream and operates in a low-power state, maintaining the current register values. Taking SC200AI as an example, there are two ways to enter sleep mode:

  1. Pulling PWDN low, at which point I2C read/write is not supported.
  2. Writing the output flow control register to 0, at which point I2C read/write is supported.
- Reset Mode

  The Sensor stops the image data stream and operates in a low-power state, resetting all registers.

  Taking SC200AI as an example, there are two ways to enter reset mode:

  1. Pulling XSHUTDN low, at which point I2C read/write is not supported.
  2. Writing the soft enable reset register to 1, this reset mode lasts for 150ns.

#### 4.1.3.1 Hardware Standby Mode

`Hardware Standby Mode` refers to the state where the Sensor is put into sleep mode by pulling down the PWDN pin, thereby operating in a low-power state, during which I2C cannot be read or written.

Taking SC200AI as an example, in DTS, the Sensor node adds the attribute rockchip,camera-module-stb = <1> to support hardware standby mode, where the attribute value indicates whether hardware standby is supported, with 1 indicating support and 0 indicating no support.

```
sc200ai: sc200ai@30 {
    compatible = "smartsens,sc200ai";
    status = "okay";
    reg = <0x30>;
    ...
    rockchip,camera-module-stb = <1>;
    port {
        sc200ai_out: endpoint {
            remote-endpoint = <&csi_dphy_input1>;
            data-lanes = <1 2>;
        };
    };
};
```

- In the Sensor driver probe interface, it is necessary to add parsing of this attribute value to determine whether hardware standby mode is supported and save it in the sc200ai->standby_hw member variable.
- If hardware standby mode is supported, the ioctl processing interface will, based on the flow state, pull up or down the relevant PWDN GPIO for RKMODULE_SET_QUICK_STREAM command. During sleep and wake-up, v4l2 ioctl commands will be issued to reset exposure, gain registers, etc. For related driver references, see 4.2.4 Driver Implementation. For hardware standby driver reference code, see the same section.

#### 4.1.3.2 Software Standby Mode

The `Software Standby Mode` refers to the method where, after waking up from hibernation, the complete Sensor register configuration is written, and then in the Sensor ioctl interface, the RKMODULE_SET_QUICK_STREAM command is processed to directly configure the sleep-wake flow registers. For related driver references, see 4.2.4 Driver Implementation and for software standby driver reference code.

### 4.1.4 Driver Implementation

Hardware standby mode driver reference code:

```
static long sc200ai_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
{
    ......
    switch (cmd) {
    case RKMODULE_SET_QUICK_STREAM:
        stream = *((u32 *)arg);

        if (sc200ai->standby_hw) { // hardware standby
            if (stream) {
                if (!IS_ERR(sc200ai->pwdn_gpio))
                    gpiod_set_value_cansleep(sc200ai->pwdn_gpio, 1);
                // Make sure __v4l2_ctrl_handler_setup can be called correctly
                sc200ai->is_standby = false;
                ret = sc200ai_write_reg(sc200ai->client, SC200AI_REG_MIPI_CTRL,
                    SC200AI_REG_VALUE_08BIT, SC200AI_MIPI_CTRL_ON);

                #if IS_REACHABLE(CONFIG_VIDEO_CAM_SLEEP_WAKEUP)
                if (__v4l2_ctrl_handler_setup(&sc200ai->ctrl_handler))
                    dev_err(&sc200ai->client->dev, "__v4l2_ctrl_handler_setup fail!");
                if (sc200ai->cur_mode->hdr_mode != NO_HDR) {
                    if (sc200ai->cam_sw_inf) {
                        ret = sc200ai_ioctl(&sc200ai->subdev,
                            PREISP_CMD_SET_HDRAE_EXP,
                            &sc200ai->cam_sw_inf->hdr_ae);
                        if (ret) {
                            dev_err(&sc200ai->client->dev,
                                "init exp fail in hdr mode\n");
                            return ret;
                        }
                    }
                }
                #endif

                ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                    SC200AI_REG_VALUE_08BIT, SC200AI_MODE_STREAMING);

                dev_info(&sc200ai->client->dev, "quickstream, streaming on: exit hw standby
mode\n");
            } else {
                ret = sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                    SC200AI_REG_VALUE_08BIT, SC200AI_MODE_SW_STANDBY);
```

```
                ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_MIPI_CTRL,
                    SC200AI_REG_VALUE_08BIT, SC200AI_MIPI_CTRL_OFF);

                if (!IS_ERR(sc200ai->pwdn_gpio))
                    gpiod_set_value_cansleep(sc200ai->pwdn_gpio, 0);

                dev_info(&sc200ai->client->dev, "quickstream, streaming off: enter hw standby
mode\n");
                sc200ai->is_standby = true;
            }
        }
    ......
}
```

Software standby driver reference implementation:

```
static long sc200ai_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
{
    ......
    switch (cmd) {
    case RKMODULE_SET_QUICK_STREAM:
        stream = *((u32 *)arg);
        ......
        else {   // software standby
            if (stream) {
                ret = sc200ai_write_reg(sc200ai->client, SC200AI_REG_MIPI_CTRL,
                    SC200AI_REG_VALUE_08BIT, SC200AI_MIPI_CTRL_ON);

                ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                    SC200AI_REG_VALUE_08BIT, SC200AI_MODE_STREAMING);
                dev_info(&sc200ai->client->dev, "quickstream, streaming on: exit soft standby
mode\n");
            } else {
                ret = sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                    SC200AI_REG_VALUE_08BIT, SC200AI_MODE_SW_STANDBY);

                ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_MIPI_CTRL,
                    SC200AI_REG_VALUE_08BIT, SC200AI_MIPI_CTRL_OFF);
                dev_info(&sc200ai->client->dev, "quickstream, streaming off: enter soft
standby mode\n");
            }
        }

        break;
    ......
    }
```

### 4.1.5 AOV Reference Documentation

During the development of AOV functionality, the following related documents can be referred to:

- Rockchip_Trouble_Shooting_Linux5.10_Camera_EN.pdf
- Rockchip_RV1106_Developer_Guide_Linux_AOV_EN.pdf

## 4.2 Synchronization Mode (Master-Slave Mode)

The control mode for a single Sensor is relatively simple, typically operating in Master mode. However, when dealing with two or more Sensors, if synchronized output flow, synchronized exposure, and other operations are required, control mode configuration becomes necessary. Generally, this is configured as a master-slave mode based on the situation.

In the following applications, we often need **multiple camera Sensors to expose and output a frame of image simultaneously, ensuring that the multi-channel images are strictly aligned in time**:

| Application Scenario | Is Multi-Sensor Synchronization Required? |
| --- | --- |
| 3D Perception / Binocular Vision / Stereo Vision | Must synchronize, otherwise depth calculation errors |
| Multi-camera Stitching / Panoramic Imaging | Frame synchronization is needed for precise image alignment |
| Machine Vision / Industrial Inspection | Multi-perspective synchronous sampling |
| Autonomous Driving / ADAS (Multi-camera Input) | Multi-channel camera timing synchronization is crucial |
| Video Conferencing / Multi-camera | ⚠ Usually not strictly required, but synchronization can enhance experience |

**Common methods to achieve multi-Sensor synchronization include:**

Using a **global synchronization signal (such as FSYNC / EFSYNC / TRIGGER / SYNC_IN)**, sent by the main control chip (e.g., SoC) to trigger signals, allowing multiple Sensors to **start exposure and read out frame data simultaneously**, thereby ensuring strict synchronization of multi-channel images in the time dimension.

### 4.2.1 Sensor's Slave Mode

In the communication and control modes of **image sensors (such as CMOS Sensor)**, **Slave mode (slave mode)** refers to the working mode where the communication, timing control, and data transmission of the Sensor are driven and controlled by an external host (such as MCU, SoC, DSP, etc.) through external signals (such as clock, synchronization signals, I2C / MIPI control). The Sensor itself does not initiate communication or control timing but passively responds to the host's instructions.

**What is FSYNC / EFSYNC?**

In a multi-Sensor system, the main control chip (such as SoC / FPGA) sends **an FSYNC pulse** to all Sensors at the same moment. After receiving this signal, all Sensors:

- **Start exposure simultaneously**

- Or **start reading out a frame of data simultaneously**
- To ensure that the timestamps and content of multiple images are synchronized.

| Synchronization Signal Name | Meaning | Explanation |
| --- | --- | --- |
| FSYNC (Frame SYNC) | Frame synchronization signal | An **externally input digital signal (usually GPIO / CMOS level)**, used to notify multiple Sensors to **start a new frame exposure or readout at the same moment** |
| EFSYNC (External Frame SYNC) | External frame synchronization signal (different naming by some manufacturers) | Similar to FSYNC, it is a signal **issued by the main control, and all Slave Sensors receive and synchronize actions** |
| TRIGGER / SYNC_IN / STROBE | Some Sensors' naming | Functionally similar, all used for externally triggered synchronization input signals |

**RK Multi-camera Synchronization Mode**:

Currently, one Sensor is configured to work in Master mode, and other Sensors work in Slave mode. After the Sensor acting as the Master completes a frame, the Slave Sensors start a frame's exposure and readout process simultaneously with the arrival of the external synchronization signal (**FSYNC / EFSYNC**), thereby ensuring the time alignment of multiple images.

### 4.2.2 Driver Implementation

1. Based on the original manufacturer's Datasheet or request the master-slave mode register configuration from the original manufacturer's FAE.

```
/* sync mode regs */
static __maybe_unused const struct regval sc200ai_internal_sync_master_start_regs[] = {
        {0x300a, 0x24}, //sync as output PAD
        {0x3032, 0xa0},
        {0x3222, 0x00}, //master mode
        {0x3216, 0x00},
        {0x3217, 0x04},
        {REG_NULL, 0x00},
};

static __maybe_unused const struct regval sc200ai_internal_sync_master_stop_regs[] = {
        {REG_NULL, 0x00},
};

static __maybe_unused const struct regval sc200ai_internal_sync_slave_start_regs[] = {
        {0x3222, 0x02}, //slave mode
        {0x3230, 0x00},
        {0x3231, 0x04}, //Rows Before Read
```

```
        {0x3224, 0x82}, //fsync trigger
        {REG_NULL, 0x00},
};

static __maybe_unused const struct regval sc200ai_internal_sync_slave_stop_regs[] = {
        {REG_NULL, 0x00},
};
```

2. Add synchronization mode variables

```
struct sc200ai {
......
enum rkmodule_sync_mode sync_mode;
......
};
```

Common synchronization mode definitions:

```
/*
 * sensor exposure sync mode
 */
enum rkmodule_sync_mode {
NO_SYNC_MODE = 0,
EXTERNAL_MASTER_MODE,
INTERNAL_MASTER_MODE,
SLAVE_MODE,
SOFT_SYNC_MODE,
};
```

Synchronization mode parsing:

Parse the synchronization mode attribute value of the relevant Sensor node in the DTS device tree to determine which synchronization mode the current Sensor is working in.

```
static int sc200ai_probe(struct i2c_client *client,
                 const struct i2c_device_id *id)
......
ret = of_property_read_string(node, RKMODULE_CAMERA_SYNC_MODE,
                              &sync_mode_name);
if (ret) {
        sc200ai->sync_mode = NO_SYNC_MODE;
        dev_err(dev, "could not get sync mode!\n");
} else {
        if (strcmp(sync_mode_name, RKMODULE_EXTERNAL_MASTER_MODE) == 0) {
                sc200ai->sync_mode = EXTERNAL_MASTER_MODE;
                dev_info(dev, "sync_mode = [EXTERNAL_MASTER_MODE]\n");
        } else if (strcmp(sync_mode_name, RKMODULE_INTERNAL_MASTER_MODE) == 0) {
                sc200ai->sync_mode = INTERNAL_MASTER_MODE;
                dev_info(dev, "sync_mode = [INTERNAL_MASTER_MODE]\n");
        } else if (strcmp(sync_mode_name, RKMODULE_SLAVE_MODE) == 0) {
                sc200ai->sync_mode = SLAVE_MODE;
                dev_info(dev, "sync_mode = [SLAVE_MODE]\n");
```

```
        } else if (strcmp(sync_mode_name, RKMODULE_SOFT_SYNC_MODE) == 0) {
                sc200ai->sync_mode = SOFT_SYNC_MODE;
                dev_info(dev, "sync_mode = [SOFT_SYNC_MODE]\n");
        } else {
                dev_info(dev, "sync_mode = [NO_SYNC_MODE]\n");
        }
}
......
```

Relevant macro definitions:

```
#define RKMODULE_CAMERA_SYNC_MODE               "rockchip,camera-module-sync-mode"
#define RKMODULE_INTERNAL_MASTER_MODE   "internal_master"
#define RKMODULE_EXTERNAL_MASTER_MODE   "external_master"
#define RKMODULE_SLAVE_MODE                             "slave"
#define RKMODULE_SOFT_SYNC_MODE                 "soft_sync"
```

3. Configure different register settings based on the current Sensor's synchronization mode when starting the stream

```
static int __sc200ai_start_stream(struct sc200ai *sc200ai)
{
int ret = 0;
......
if (!sc200ai->is_thunderboot) {
        ret = sc200ai_write_array(sc200ai->client, sc200ai->cur_mode->reg_list);
        if (ret)
                return ret;

        ......

        if (sc200ai->sync_mode == INTERNAL_MASTER_MODE ||
            sc200ai->sync_mode == SOFT_SYNC_MODE) {
                ret |= sc200ai_write_array(sc200ai->client,
                                           sc200ai_internal_sync_master_start_regs);
        } else if (sc200ai->sync_mode == EXTERNAL_MASTER_MODE) {
                ret |= sc200ai_write_array(sc200ai->client,
                                           sc200ai_internal_sync_slave_start_regs);
        } else if (sc200ai->sync_mode == SLAVE_MODE) {
                ret |= sc200ai_write_array(sc200ai->client,
                                           sc200ai_internal_sync_slave_start_regs);
        }
        if (ret) {
                dev_err(&sc200ai->client->dev,
                        "write sync regs failed\n");
                return ret;
        }
}

if (sc200ai->sync_mode == NO_SYNC_MODE)
        ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                                 SC200AI_REG_VALUE_08BIT, SC200AI_MODE_STREAMING);
```

```
return ret;
}
```

4. Set/Get synchronization mode in RKAIQ

   If the device tree does not configure the synchronization working mode of the relevant Sensor, the synchronization working mode of multiple cameras can also be set through the ioctl command by calling the rk_aiq interface.

```
static long sc200ai_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
{
......
case RKMODULE_GET_SYNC_MODE:
        sync_mode = (u32 *)arg;
        *sync_mode = sc200ai->sync_mode;
        break;
case RKMODULE_SET_SYNC_MODE:
        sync_mode = (u32 *)arg;
        if (sync_mode) {
                sc200ai->sync_mode = *sync_mode;
                dev_info(&sc200ai->client->dev, "set sync mode is: %s\n",
                         ((*sync_mode == EXTERNAL_MASTER_MODE) ||
                          (*sync_mode == SLAVE_MODE)) ? "secondary" : "primary");
        } else {
                dev_info(&sc200ai->client->dev, "set sync mode is: NO_SYNC_MODE\n");
        }
        break;
......
}
```

## 4.3 Flashlight Control

In the Sensor driver, control over the flashlight can be implemented, with supported control types including PWM and GPIO. The application layer manages the process of turning the flashlight on and off. Proper configuration of relevant parameters is required.

### 4.3.1 Driver Implementation

1. Header File

```
#include "light_ctl.h"
```

2. Add Relevant Control Variables

```
struct xxxx {
......
bool                                    enable_light_ctl;
struct rk_light_param   light_param;
......
};
```

3. Add ioctl RKCIS_CMD_FLASH_LIGHT_CTRL Command Handling

```
static long xxxx_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
......
struct rk_light_param *light_param;
......
case RKCIS_CMD_FLASH_LIGHT_CTRL:
        dev_info(&xxxx->client->dev, "set flash light param\n");
        light_param = (struct rk_light_param *)arg;
        if (light_param->light_enable) {
                memcpy(&xxxx->light_param, light_param, sizeof(struct rk_light_param));
                xxxx->enable_light_ctl = true;
        } else {
                xxxx->enable_light_ctl = false;
        }
        ret = light_ctl_write(xxxx->module_index, light_param);
        break;

......
};

static long xxxx_compat_ioctl32(struct v4l2_subdev *sd,
                         unsigned int cmd, unsigned long arg)
{
......
        struct rk_light_param *light_param;
......
        switch (cmd) {
        ......
        case RKCIS_CMD_FLASH_LIGHT_CTRL:
            light_param = kzalloc(sizeof(*light_param), GFP_KERNEL);
            if (!light_param) {
                ret = -ENOMEM;
                return ret;
            }

            ret = copy_from_user(light_param, up, sizeof(*light_param));
            if (!ret)
                ret = mis5001_ioctl(sd, cmd, light_param);
            else
                ret = -EFAULT;
            kfree(light_param);
            break;
        }
```

```
.....
}
```

### 4.3.2 Reference Documents

For specific implementation and details, refer to the following documents:

`Rockchip_Trouble_Shooting_Linux5.10_Camera_EN.pdf` Section **7.2 How to Control the Fill Light**.

## 4.4 Resolution Switching

Customers may use different sensor resolutions in various scenarios within their applications. To accommodate this, the sensor driver supports commands from the upper-layer applications to switch the sensor resolution.

### 4.4.1 Driver Implementation

Driver code reference:

```
static int sc200ai_set_setting(struct sc200ai *sc200ai, struct rk_sensor_setting *setting)
{
        int i = 0;
        int cur_fps = 0;
        s64 h_blank, vblank_def;
        u64 pixel_rate = 0;
        const struct sc200ai_mode *mode = NULL;
        const struct sc200ai_mode *match = NULL;

        dev_info(&sc200ai->client->dev,
                "sensor setting: %d x %d, fps:%d fmt:%d, mode:%d\n",
                setting->width, setting->height,
                setting->fps, setting->fmt, setting->mode);

        for (i = 0; i < ARRAY_SIZE(supported_modes); i++) {
                mode = &supported_modes[i];
                if (mode->width == setting->width &&
                    mode->height == setting->height &&
                    mode->hdr_mode == setting->mode &&
                    mode->bus_fmt == setting->fmt) {
                        cur_fps = DIV_ROUND_CLOSEST(mode->max_fps.denominator, mode-
>max_fps.numerator);
                        if (cur_fps == setting->fps) {
                                match = mode;
                                break;
                        }
                }
        }
```

```
        if (match) {
                dev_info(&sc200ai->client->dev, "-----%s: match the support mode, mode
idx:%d-----\n",
                        __func__, i);
                sc200ai->cur_mode = mode;

                h_blank = mode->hts_def - mode->width;
                __v4l2_ctrl_modify_range(sc200ai->hblank, h_blank,
                                         h_blank, 1, h_blank);
                vblank_def = mode->vts_def - mode->height;
                __v4l2_ctrl_modify_range(sc200ai->vblank, vblank_def,
                                         SC200AI_VTS_MAX - mode->height,
                                         1, vblank_def);
                __v4l2_ctrl_s_ctrl(sc200ai->link_freq, mode->mipi_freq_idx);
                pixel_rate = (u32)link_freq_menu_items[mode->mipi_freq_idx] /
                             mode->bpp * 2 * SC200AI_LANES;
                __v4l2_ctrl_s_ctrl_int64(sc200ai->pixel_rate, pixel_rate);
                dev_info(&sc200ai->client->dev, "freq_idx:%d pixel_rate:%lld\n",
                        mode->mipi_freq_idx, pixel_rate);

                sc200ai->cur_vts = mode->vts_def;
                sc200ai->cur_fps = mode->max_fps;

                dev_info(&sc200ai->client->dev, "hts_def:%d cur_vts:%d cur_fps:%d\n",
                        mode->hts_def, mode->vts_def,
                        sc200ai->cur_fps.denominator / sc200ai->cur_fps.numerator);
        } else {
                dev_err(&sc200ai->client->dev, "couldn't match the support modes\n");
                return -EINVAL;
        }

        return 0;
}

static long sc200ai_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
{
        ......
        struct rk_sensor_setting *setting;
        ......

        switch (cmd) {
        ......
        case RKCIS_CMD_SELECT_SETTING:
                setting = (struct rk_sensor_setting *)arg;
                ret = sc200ai_set_setting(sc200ai, setting);
                break;

        ......
        }
}
```

## 4.5 Compatibility of Multi-Lane Configurations

Sensors may require compatibility with different Lane numbers of register sequence configurations based on the differences in the main control platform, quantity, performance, and connection methods. Therefore, the driver has been designed to accommodate this situation.

### 4.5.1 Driver Implementation

Specific code reference is as follows:

```
// 4 Lane Register Configuration Support List
static const struct sc450ai_mode supported_modes_4lane[] = {
    {
        ...
    },
    {
        ...
    },
};

// 2 Lane Register Configuration Support List
static const struct sc450ai_mode supported_modes_2lane[] = {
    {
        ...
    },
    {
        ...
    },
}
static int sc450ai_probe(struct i2c_client *client,
                         const struct i2c_device_id *id)
{
    struct device_node *node = dev->of_node;
        struct device_node *endpoint;
    ......
        endpoint = of_graph_get_next_endpoint(dev->of_node, NULL);
        if (!endpoint) {
                dev_err(dev, "Failed to get endpoint\n");
                return -EINVAL;
        }
        ret = v4l2_fwnode_endpoint_parse(of_fwnode_handle(endpoint),
                &sc450ai->bus_cfg);
        of_node_put(endpoint);
        if (ret) {
                dev_err(dev, "Failed to get bus config\n");
                return -EINVAL;
        }

    // Based on the lane number configured in dts, select different register support list
arrays
        if (sc450ai->bus_cfg.bus.mipi_csi2.num_data_lanes == 4) {
                sc450ai->supported_modes = supported_modes_4lane;
                sc450ai->cfg_num = ARRAY_SIZE(supported_modes_4lane);
```

```
                dev_info(dev, "detect sc450ai lane: %d\n",
                        sc450ai->bus_cfg.bus.mipi_csi2.num_data_lanes);
        } else {
                sc450ai->supported_modes = supported_modes_2lane;
                sc450ai->cfg_num = ARRAY_SIZE(supported_modes_2lane);
                dev_info(dev, "detect sc450ai lane: %d\n",
                        sc450ai->bus_cfg.bus.mipi_csi2.num_data_lanes);
        }
    ......
}
```

Support for 2Lane or 4Lane `supported_modes` is determined during the probe interface call of the Sensor, by parsing the Lane count of the Sensor in DTS for initialization. The parsing result is stored in `bus_cfg.bus.mipi_csi2.num_data_lanes`, and the total number of `cfg_num` for each Lane mode is configured.

# 5. FAQ

## 5.1 Sensor Not Recognized, Probe Failure, I2C Communication Failure

If the Sensor ID is not recognized, it has no relation to RKISP or RKCIF; it simply means the power-up sequence for the Sensor has not been met. Please troubleshoot in the following order:

1. Verify if the 7-bit I2C slave ID of the Sensor is correct and has not been mistakenly written as an 8-bit ID.
2. Check if MCLK is outputting and if the voltage amplitude is correct. MCLK is generally 24MHz, but it can also be 27MHz.
3. Ensure that the power-up sequence for the Sensor meets the requirements, which mainly includes checking if the avdd, dovdd, dvdd voltages are normal and comply with the Sensor manual requirements, and whether the PWDN, RESET GPIO pins are configured correctly, etc.

### 5.1.1 Measuring MCLK and VDD Voltages Unsuccessfully

In the implementation of Sensor drivers, MCLK and power are typically enabled only when necessary, hence they are default disabled upon power-on.

For debugging, you can comment out the implementation of the power_off() function in the driver, preventing power-down and facilitating measurement.

When using an oscilloscope, check if the bandwidth is sufficient, with a recommendation of at least 48M bandwidth.

1. The Sensor has not correctly enabled MCLK; refer to the operations on MCLK in drivers/media/i2c/ov5695.c.
2. The gpio is occupied by another module, in which case the kernel log will generally provide corresponding hints. You can also check if the pin-ctrl register settings are correct using the io command.

### 5.1.2 Verify if the Power-Up Sequence of the Sensor Meets the Requirements

The Datasheet of a Sensor typically provides a detailed description of the power-up sequence and interval requirements for each power supply. Please check whether these requirements are met using an oscilloscope. Some Sensors, such as the ov5695, do not have a specific sequence requirement for their vdd power supply during power-up, and their drivers may use a regulator_bulk to manage the power supply. However, others, like ov2685.c, have specific sequence requirements, and their drivers use multiple regulators to control the power supply, such as avdd_regulator and dovdd_regulator. Please select based on the actual situation.

## 5.2 What should be the default values for `exp_def`, `hts_def`, and `vts_def` in the driver?

If you have the contact information of the Sensor manufacturer, please contact them to obtain it. Otherwise, refer to the Datasheet to find the corresponding registers and look up the configured values during initialization. Taking SC200AI as an example:

```
#define SC200AI_REG_VTS_H               0x320e
#define SC200AI_REG_VTS_L               0x320f
...
static const struct regval sc200ai_linear_10_1920x1080_30fps_regs[] = {
        ...
        //HTS=1100*2=2200
        {0x320c, 0x04},
        {0x320d, 0x4c},
        //VTS=1125
        {0x320e, 0x04},
        {0x320f, 0x65},
        ...
};

static const struct sc200ai_mode supported_modes[] = {
        {
                .width = 1920,
                .height = 1080,
                .max_fps = {
                        .numerator = 10000,
                        .denominator = 300000,
                },
                .exp_def = 0x0080,      // exp exposure default value, generally should be
less than the vts default value, maximum vts_def - 8 (exposure lines, the specific amount to
subtract, refer to the Sensor manual, may be 4/8/10)
                .hts_def = 0x44C * 2,// hts default value filled in with the default value
given in the corresponding register list (can be multiplied)
                .vts_def = 0x0465,      // vts default value filled in with the default value
given in the corresponding register list
                ...
        },
```

```
        ...
};
```

## 5.3 What should be the values for `link_freq` and `pixel_rate`

`link_freq` refers to the actual frequency of the MIPI Clk. Note that it is not the 24M MCLK, but the MIPI dn/dp clk. It is preferable to inquire through the original factory window or look up the Datasheet for any related parameters. In general, the actual value of `link_freq` will not be less than the result calculated by the following formula, with the unit being (Hz):

```
link_freq = width * height * fps * bits_per_pixel / lanes / 2
```

If the actual value of `link_freq` is not known, it can be measured with an oscilloscope.

`pixel_rate` refers to the number of pixels transmitted per second. Once `link_freq` is determined, it can be calculated using the following formula:

```
pclk = link_freq * 2 * lanes / bits_per_pixel
```

## 5.4 How to Verify if the Sensor is Successfully Brought up

First, it is necessary to confirm whether the Sensor ID can be recognized, that is, whether there are any anomalies in I2C reading and writing. At this time, using `media-ctl -p -d /dev/media0` should display specific information about the Sensor, such as name and resolution. As shown below:

```
# media-ctl -p -d /dev/media0 | tail -n 30
                -> "rkcif_tools_id2":0 []
        pad11: Source
                -> "stream_cif_mipi_id0":0 []
                -> "stream_cif_mipi_id1":0 []
                -> "stream_cif_mipi_id2":0 []
                -> "stream_cif_mipi_id3":0 []
                -> "rkcif_scale_ch0":0 []
                -> "rkcif_scale_ch1":0 []
                -> "rkcif_scale_ch2":0 []
                -> "rkcif_scale_ch3":0 []
                -> "rkcif_tools_id0":0 []
                -> "rkcif_tools_id1":0 []
                -> "rkcif_tools_id2":0 [ENABLED]

- entity 58: rockchip-csi2-dphy0 (2 pads, 2 links)
             type V4L2 subdev subtype Unknown flags 0
             device node name /dev/v4l-subdev1
        pad0: Sink
                [fmt:SBGGR10_1X10/1920x1080@10000/300000 field:none]
                <- "m00_b_sc200ai 4-0030":0 [ENABLED]
        pad1: Source
```

```
                -> "rockchip-mipi-csi2":0 [ENABLED]

- entity 63: m00_b_sc200ai 4-0030 (1 pad, 1 link)
             type V4L2 subdev subtype Sensor flags 0
             device node name /dev/v4l-subdev2
        pad0: Source
                [fmt:SBGGR10_1X10/1920x1080@10000/300000 field:none]
                -> "rockchip-csi2-dphy0":0 [ENABLED]
```

If `media-ctl` or `v4l2-ctl` is not compiled and packaged in the system, it is necessary to use `dmesg` to print the kernel log and check whether the relevant Sensor is detected successfully.

Secondly, when capturing images at the upper layer, MIPI must be able to output data without reporting any MIPI / ISP related errors, and the application layer should be able to receive frames.

# 5.5 Sensor Streaming with CRC/ECC Errors

## 5.5.1 Confirming Error Sources and Log Details

**Locating Logs:**

- First, precisely search for error messages containing keywords such as `ERR1`, `ERR2`, `CRC`, `ECC`, `mipi`, `csi2`, `phy` in `dmesg` or `kernel log`.
- The CSI/PHY driver for Rockchip typically prints detailed register states when errors occur. **Focus on recording the error type codes for `ERR1`, `ERR2`** and the **`VC (Virtual Channel)`** numbers.
- Typical error log snippet:

  ```
  [  126.741234] rockchip-mipi-csi2 feaa0000.csi2-dphy: ERR1: 0x00000040 (ECC Error, VC1),
  FIFO overflow? Check timings!
  [  126.741259] rockchip-mipi-csi2 feaa0000.csi2-dphy: ERR2: 0x00000010 (CRC Error, VC0)
  ```

**Understanding Error Types:**

- **`ERR1` (CSIS_ERR1)**: Often indicates **protocol layer errors** (ECC, frame format errors, packet header check errors).
- **`ERR2` (CSIS_ERR2)**: Often indicates **transport layer errors** (FIFO overflow at the SoC end, data receive timeout, synchronization code errors).
- **`CRC Error`**: Data packet checksum error (bit flips during transmission).
- **`ECC Error`**: Failed detection and correction of packet header errors (usually more serious, indicating physical layer issues).
- **`Overflow`**: Overflow of the receiver's FIFO (usually caused by timing mismatch).

## 5.5.2 Software Configuration Check (Driver and DTS)

**Sensor Output Timing Matching:**

- Verify that the `HTS (Total Width)` and `VTS (Total Height)` settings in the Sensor driver **are completely consistent with the SoC receiver side (CSI2-DPHY Controller) configuration**. Inconsistencies can lead to FIFO overflow (`ERR2 Overflow`). Use the following commands to verify:

```
media-ctl -p -d /dev/mediaX  # View the links between Entities and current format
v4l2-ctl -d /dev/v4l-subdevY --get-fmt-video # View the Sensor output format
v4l2-ctl -d /dev/v4l-subdevZ --get-fmt-video # View the CSI-DPHY input format
```

- Ensure that the `MIPI Pixel Clock (Sensor output rate)` is less than or equal to the `maximum rate supported by the SoC CSI-DPHY` (RK3566: ~2.5Gbps/lane, RK3588: ~2.5Gbps/lane).

**VC (Virtual Channel) Matching:**

- Check that the MIPI VC number set in the Sensor driver (e.g., `0`, `1`) **must be consistent with the `csi2_dphy` or `rkcif_mipi_lvds` receiver VC number configured in the DTS**. Inconsistencies can lead to CRC errors.

**DTS(Device Tree) Configuration:**

- Check the csi2_dphy node configuration

```
csi2_dphy_hw: csi2-dphy@fedc0000 {
    compatible = "rockchip,rk3588-csi2-dphy-hw";
    clocks = ...; // Clock configuration must be correct
    resets = ...;
    rockchip,hw = <&csi2_dphy0>; // Hardware version
};
csi2_dphy0: csi2-dphy0 {
    compatible = "rockchip,rk3568-csi2-dphy";
    phys = <&mipi_dphy0>; // Point to the correct PHY
    phy-names = "dphy";
    ports {
        port@0 {
            reg = <0>;
            #address-cells = <1>;
            #size-cells = <0>;
            // Key point: CSI receiver VC setting
            endpoint {
                remote-endpoint = <&sensor_out>;
                data-lanes = <1 2 3 4>; // Lane count must match Sensor
                virtual-channel-index = <0>; // Must be equal to Sensor output VC
            };
        };
    };
};
```

- Check clocks and power supply
  - Ensure that the clock frequency configuration for the Sensor, MIPI DPHY, and CSI2 Host Controller is correct and without conflict.
  - The power supply for the Sensor (`DOVDD`, `AVDD`, `DVDD`) voltage must be within the specification range, ripple < 50mV (measured with an oscilloscope).

## 5.5.3 Hardware Link Troubleshooting (Physical Layer Issues Account for Over 90%!)

- **Physical Connection Reliability:**
  - **FPC Flex Cable Removal and Reinsertion:** Reinsert the FPC flexible cable, ensuring the connector is securely locked in place. **Replace** with a known good FPC cable for testing.
  - **Contact Point Cleaning:** Inspect the connector's gold fingers for oxidation or contamination and treat with precision electronic cleaner.
  - **Impedance Matching:** Ensure the cable impedance is the standard **100Ω ±10%** (differential impedance). Inferior cables are a common source of errors.
- **Signal Integrity Testing (Essential Tools: Oscilloscope/Protocol Analyzer):**
  - **Power Supply Ripple Measurement:** Use an oscilloscope to measure noise on all power pins of the Sensor (especially 1.2V/1.8V DVDD) with noise <50mV pk-pk.
  - **MIPI Differential Signal Quality:**

  Measure at the CLK+/CLK- and DATA+/DATA- test points on the Sensor side:
  - **Eye Diagram:** Eye height ≥ 100mV, eye width ≥ 0.4 UI, with clear and monotonic edges. A blurry/closed eye diagram is a critical issue.
  - **Slew Rate:** Should be within the range of 1.5~4V/ns (based on D-PHY version).
  - **Overshoot/Ringing:** < 20% Vdiff pp.
  - **Common Mode Noise:** < 100mV.
  - **Skew:** Data-to-Clk Skew < 0.3 UI.
  - **Low-Speed Mode Verification:** Attempt to configure the Sensor to MIPI low-speed mode (e.g., reduced to 500Mbps) to see if the errors disappear.
- **Reference Clock (MCLK) Quality:**
  - Measure the MCLK signal input to the Sensor (usually 24MHz or 27MHz) for **frequency stability (±50ppm)**, **phase jitter (< 50ps RMS)**.
  - Oscilloscope measurement requirements: Use shielded cables, minimize ground loops.
- **Nodes and Interference:**
  - **Ground Loop Impedance:** Impedance between Sensor board ground and SoC board ground < 1Ω.
  - **Interference Source Isolation:** Keep away from high-frequency circuits (WiFi/DC-DC), motors, and strong magnetic fields.

## 5.5.4 System-Level Verification and Debugging

**Lane Count and Polarity Check:**

- Ensure that the `data-lanes` sequence and count configured in DTS are consistent with the actual physical Lanes used.
- **Special Attention: Lane Polarity Inversion.** MIPI allows for internal polarity inversion on the RX end; check if DTS has a setting for `lane-polarities`:

**Resolution/Frame Rate Reduction Testing:**

- Reduce the Sensor output resolution to the lowest (e.g., 320x240 @10fps) to decrease data transfer volume, **if the error disappears, it indicates a hardware bandwidth bottleneck (signal quality issue)**.

**Using Test Patterns:**

- Enable the built-in test pattern mode of the Sensor (Color Bar/Checkerboard), **bypassing the sensor's analog circuitry to directly test MIPI output**.

**Debugging Tools:**

- **RK ISP Debugging Tool:** Activate `rkisp_3A_server` in `/etc/init.d/` to view detailed logs.
- **v4l2-ctl Monitoring:**

```
v4l2-ctl -d /dev/video0 --stream-mmap --stream-count=100 --stream-skip=100
```

**Temperature Factors:**

- High temperatures can degrade signal quality. Observe whether the error rate worsens with increasing temperature (environmental temperature > 85°C).

## 5.5.5 Emergency Situation Handling Strategy

1. **Priority Decrease Speed:** Immediately reduce the MIPI rate to less than 1.5Gbps/lane for testing.
2. **Minimize Lanes:** Attempt to transmit data using only 1 Lane.
3. **Isolate Interference:** Remove all irrelevant peripherals and reduce the frequency of CPU/GPU.
4. **Replacement Testing:** Replace the Sensor module, core board, or motherboard (especially when suspecting PCB impedance issues).

## 5.5.6 Typical Repair Measures

**Physical Layer Issues (90%):**

- Replace with high-quality MIPI cables (impedance control 100Ω±5%).
- Optimize PCB Layout: Shorten the length of differential lines, eliminate via stubs, and increase the spacing between differential pairs.
- Add terminal resistors (50Ω to ground).

**Software Configuration Issues (9%):**

- Precisely match timing configurations: `VTS`, `HTS`, `Pixel Clock`.
- Correct VC index and Lane Map.
- Update driver or firmware versions (MIPI PHY firmware is particularly important).

**Hardware Design Deficiencies (1%):**

- Redesign the power tree, adding LDO filtering.
- Replace with high-precision crystal oscillators or clock buffers.
- Enhance ESD protection level (selecting TVS diodes).

## 5.5.7 Precautions for Sensor Index

The index of the Sensor is filled in the DTS with `rockchip,camera-module-index`.

For the convenience of device lookup, librkaiq and rockit prioritize supporting the Sensor's index as the device index. The tool_server that comes with the tuning tools also searches for devices based on the Sensor index. If a tool similar to this is used without specifying the device number with -d, the default is 0.

Therefore, it is recommended that:

- For single-camera setups, set the sensor index to 0.
- For dual-camera setups, set the sensor index to 0/1.
- For multi-camera setups, set the sensor index to 0/1/2/3...

# 6. AOV Reference Documents

Writing the Sensor driver code is just one step in Camera debugging. When debugging, encountering different platform device tree configurations, specific product forms (Quick Start, AOV), or image output issues during development, the following development documents can be referenced:

1. Rockchip_Driver_Guide_VI_EN.pdf, last ver: v1.2.0
2. Rockchip Quick Start For Camera.pdf, last ver: v1.0.0
3. Rockchip_Trouble_Shooting_Linux5.10_Camera_EN.pdf
4. Rockchip_Developer_Guide_Linux_AOV_EN.pdf

# 7. Appendix

## 7.1 CIS Driver V4L2-controls List

| CID | Description |
| --- | --- |
| V4L2_CID_VBLANK | Vertical blanking. The idle period after every frame during which no image data is produced. The unit of vertical blanking is a line. Every line has a length of the image width plus horizontal blanking at the pixel rate defined by V4L2_CID_PIXEL_RATE control in the same sub-device. |
| V4L2_CID_HBLANK | Horizontal blanking. The idle period after every line of image data during which no image data is produced. The unit of horizontal blanking is pixels. |
| V4L2_CID_EXPOSURE | Determines the exposure time of the camera sensor. The exposure time is limited by the frame interval. |
| V4L2_CID_ANALOGUE_GAIN | Analogue gain is gain affecting all colour components in the pixel matrix. The gain operation is performed in the analogue domain before A/D conversion. |
| V4L2_CID_PIXEL_RATE | Pixel rate in the source pads of the subdev. This control is read-only and its unit is pixels / second. For example, on a mipi bus: pixel_rate = link_freq * 2 * nr_of_lanes / bits_per_sample |
| V4L2_CID_LINK_FREQ | Data bus frequency. Together with the media bus pixel code, bus type (clock cycles per sample), the data bus frequency defines the pixel rate (V4L2_CID_PIXEL_RATE) in the pixel array (or possibly elsewhere, if the device is not an image sensor). The frame rate can be calculated from the pixel clock, image width and height, and horizontal and vertical blanking. While the pixel rate control may be defined elsewhere than in the subdev containing the pixel array, the frame rate cannot be obtained from that information. This is because only on the pixel array it can be assumed that the vertical and horizontal blanking information is exact: no other blanking is allowed in the pixel array. The selection of frame rate is performed by selecting the desired horizontal and vertical blanking. The unit of this control is Hz. |

## 7.2 MEDIA_BUS_FMT List

| CIS Sensor Type | Sensor Output Format |
| --- | --- |
| Bayer RAW | MEDIA_BUS_FMT_SBGGR10_1X10<br>MEDIA_BUS_FMT_SRGGB10_1X10<br>MEDIA_BUS_FMT_SGBRG10_1X10<br>MEDIA_BUS_FMT_SGRBG10_1X10<br>MEDIA_BUS_FMT_SRGGB12_1X12<br>MEDIA_BUS_FMT_SBGGR12_1X12<br>MEDIA_BUS_FMT_SGBRG12_1X12<br>MEDIA_BUS_FMT_SGRBG12_1X12<br>MEDIA_BUS_FMT_SRGGB8_1X8<br>MEDIA_BUS_FMT_SBGGR8_1X8<br>MEDIA_BUS_FMT_SGBRG8_1X8<br>MEDIA_BUS_FMT_SGRBG8_1X8 |
| YUV | MEDIA_BUS_FMT_YUYV8_2X8<br>MEDIA_BUS_FMT_YVYU8_2X8<br>MEDIA_BUS_FMT_UYVY8_2X8<br>MEDIA_BUS_FMT_VYUY8_2X8<br>MEDIA_BUS_FMT_YUYV10_2X10<br>MEDIA_BUS_FMT_YVYU10_2X10<br>MEDIA_BUS_FMT_UYVY10_2X10<br>MEDIA_BUS_FMT_VYUY10_2X10<br>MEDIA_BUS_FMT_YUYV12_2X12<br>MEDIA_BUS_FMT_YVYU12_2X12<br>MEDIA_BUS_FMT_UYVY12_2X12<br>MEDIA_BUS_FMT_VYUY12_2X12 |
| Only Y(Black and White) i.e., raw bw sensor | MEDIA_BUS_FMT_Y8_1X8<br>MEDIA_BUS_FMT_Y10_1X10<br>MEDIA_BUS_FMT_Y12_1X12 |

## 7.3 Camera Reference Driver List

| CIS Data Interface | CIS Output Data Type | Frame/Field | Reference Driver |
| --- | --- | --- | --- |
| MIPI | Bayer RAW | frame | **0.3M**<br>ov7750.c<br>gc0403.c<br><br>**0.9M**<br>jx_h62.c<br>jx_h63p<br><br>**1.2M**<br>ov9750.c<br>jx-h65.c<br>mia1321.c<br><br>**2M**<br>jx_f37.c<br>mis20s1.c<br>mis2031.c<br>ov2685.c<br>ov2680.c<br>ov2735.c<br>ov02g10.c<br>ov02b10.c<br>gc2385.c<br>gc2355.c<br>gc2053.c<br>gc2093.c<br>sc2232.c<br>sc2239.c<br>sc223a.c<br>sc2310.c<br>sc2336.c<br>sc2336p.c<br>sc230ai.c<br>sc231hai.c<br>sc235hai.c<br>sc210iot.c<br>sp250a.c<br><br>**4M**<br>gc4c33.c<br>jx_k04.c<br>jx_k17.c<br>mis4001.c<br>mis40c1.c<br>mis40h1.c<br>ov4686.c |

| CIS Data Interface | CIS Output Data Type | Frame/Field | Reference Driver |
| --- | --- | --- | --- |
| | | | ov4688.c<br>ov4689.c<br>os04c10.c<br>os04a10.c<br>sc401ai.c<br>sc4236.c<br>sc450ai.c<br>sc485sl.c<br>imx347.c<br><br>**5M**<br>mis5001.c<br>os05a20.c<br>ov5695.c<br>ov5648.c<br>ov5670.c<br>gc5024.c<br>gc5025.c<br>gc5035.c<br>hm5040.c<br>sc5239.c<br>hynix_hi556.c<br><br>**8M**<br>ar0822.c<br>gc8613.c<br>mis8001.c<br>ov8858.c<br>os08a10.c<br>os08a20.c<br>sc8220.c<br>sc830ai.c<br>sc835hai.c<br>sc850sl.c<br>imx378.c<br>imx317.c<br>imx219.c<br>gc8034.c<br>hynix_hi846.c<br>s5kgm1sp.c<br>s5k4h7yx.c<br><br>**13M**<br>ov13850.c<br>ov13b10.c |

| CIS Data Interface | CIS Output Data Type | Frame/Field | Reference Driver |
| --- | --- | --- | --- |
| | | | imx258.c<br>ov12d2q.c |
| MIPI | Bayer raw hdr | frame | **2M**<br>imx307.c<br>imx327.c<br>imx462.c<br>gc2093.c<br>ov02k10.c<br>ov2718.c<br>sc200ai.c<br>sc2310.c<br>jx-f37.c<br><br>**4M**<br>ov4689.c<br>os04a10.c<br>imx347.c<br>imx464.c<br>sc4238.c<br><br>**5M**<br>imx335.c<br>os05a20.c<br>sc500ai.c<br><br>**8M**<br>imx334.c<br>imx415.c |
| MIPI | YUV | frame | 2M<br>gc2145.c |
| MIPI | RAW BW | frame | **0.3M**<br>ov7251.c<br><br>**1M**<br>ov9281.c<br><br>**1.3M**<br>sc132gs.c |

| CIS Data Interface | CIS Output Data Type | Frame/Field | Reference Driver |
| --- | --- | --- | --- |
| MIPI | YUV | field | tc35874x.c |
| ITU.BT60 | Bayer RAW | | **2M**<br>imx323.c<br>ar0230.c |
| ITU.BT601 | YUV | | **0.3M**<br>gc0329.c<br>gc0312.c<br>gc032a.c<br><br>**2M**<br>gc2145.c<br>gc2155.c<br>gc2035.c<br>bf3925.c |
| ITU.BT601 | RAW BW | | **0.3M**<br>sc031gs.c<br><br>**1.3M**<br>sc032gs.c |
| ITU.BT656 | Bayer RAW | | **2M**<br>imx323.c(supported) |